C4 Predict–Test–Explain Workbook

Owner: Stanley Young **Act phase:** Day 1 = 2026-08-20 → Day 10 = 2026-09-04. Ten **working** days: Aug 20, 21, 24, 25, 27, 28, 31, Sep 1, 3, 4. **Rule:** predictions are written BEFORE the experiment runs. Results after. Never edit a prediction once the test has run — a wrong prediction is the evidence, not a mistake.

> This is the single living workbook for C4. It is updated in place, never regenerated.

How to use this

Each experiment has three fields:

FIELD	WHEN
My prediction	Before the test
Actual result	After the test
The gap	After the test

Predictions are drafted before the experiment runs and are never edited afterwards. Revise them freely up until the moment you run the test — after that they are the record.

The gap is the deliverable. "Prediction correct" is a fine outcome but a cheap one. "Prediction wrong, and here is why my mental model was off" is the sentence that goes in the final evidence.

Cycle map

CYCLE	DAYS	DATES	LEARNING QUESTION	STATUS
L1 · Staying alive	D1–D2	**Aug 20–21**	Why does a watch app stop recording, and what does `HKWorkoutSession` change?	**COMPLETE** · E1.0 ✔ E1.1 ✔ E1.2 ✔ E1.3 ✔ · gate passed Day 2 · E1.3 raises an open question for L2
L2 · Recoverability	D3–D4	**Aug 24–25**	What has to be true for a workout to survive the app dying?	**COMPLETE** · E2.0 ✔ E2.0b ✔ E2.1 ✔ E2.2 ✔ E2.2b ✔ E2.3 ✔ · endpoint mystery solved, atomicity proven
L3 · The crossing	D5–D6	**Aug 27–28**	Does a workout actually cross to the phone unaided, and what arrives when it does?	**COMPLETE** · E3.0 ✔ E3.1 ✔ E3.2 ✔ · L4 needs rescoping
L4 · The limits of the free crossing	D7–D8	**Aug 31 – Sep 1**	Where does the free crossing break, and what genuinely still needs a transport of my own?	**COMPLETE** · E4.0 ✔ · E4.1 not needed · E4.2 ✔ no transport to be built
L5 · Honest representation	D9–D10	**Sep 3–4**	What can this data honestly say, and what can it not?	**COMPLETE** · E5.0 ✔ (5/5) E5.1 ✔ E5.2 ✔ · found a fifth category, ASSUMED
L6 · The platform's own idiom	–	**Sep 3**	Is the way I record structure the way the platform records it?	**COMPLETE** · E6.0 ✔ E6.1 ✔ E6.2 ✔ E6.3 ✔ · no predictions written, recorded as a departure

Re-baselined 2026-08-24 against the project Gantt. The ten Act days are **working days, not consecutive calendar days**: Aug 20, 21, 24, 25, 27, 28, 31, Sep 1, 3, 4. Weekends and the intervening gap days are not Act days.

This means the project is ON SCHEDULE, not behind. An earlier note in this workbook assumed consecutive days and concluded L2 was running two days late. That was wrong: 24 August is Day 3, the first day of L2, exactly as planned.

Cycle L6 has been dropped. The Gantt carries five cycles, not six. Presentation preparation now sits inside L5 or after Act ends on 4 September, rather than consuming a numbered day of its own.

CYCLE L1 · Staying alive — Days 1–2 (Aug 20–21)

Learning question: Why does a watch app stop recording, and what does `HKWorkoutSession` actually change?

Gate: if E1.2 has not passed by end of Day 2, cut to the floor scope and record why. That is a finding, not a failure.

E1.0 — Foundation: can I even talk to HealthKit on real hardware?

Run: Day 1

Scaffold status (Day 1, 09:20): project generated and verified. `HyroxCoach.xcodeproj` built by XcodeGen from `project.yml`. Both targets carry the HealthKit entitlement, both Info.plists carry the usage descriptions, watch app is a modern single-target `WKApplication` embedded in the iOS app.

Verified by `plutil -lint` and by reading the generated pbxproj. **Not yet built or run** — blocked on the `xcode-select` fix below.

My prediction

1. Both targets need the HealthKit capability added separately. Adding it to the iOS target alone will not enable it on the watch.
2. `NSHealthShareUsageDescription` and `NSHealthUpdateUsageDescription` are required. Missing either causes a **crash at authorisation time**, not a build error — so it will look like a runtime bug, not a config mistake.
3. `HKHealthStore.isHealthDataAvailable()` returns `true` on both the watch and the iPhone.
4. The authorisation sheet appears **on the watch** when requested from the watch app.
5. `requestAuthorization` succeeds even if the user denies — it reports that the *request completed*, not that permission was granted. Checking `authorizationStatus` for a read type will NOT reliably tell me whether read access was granted (Apple deliberately hides read denial to prevent inference).
6. Most likely time sink today: provisioning and signing across two targets, not HealthKit itself.

Actual result

Run 2026-08-20, ~11:03, Apple Watch Ultra 3 (watchOS 26.6), iPhone 16 Pro Max (iOS 26.6.1), Xcode 26.6.

- **PASS.** Round trip succeeded: wrote 1 kcal `activeEnergyBurned`, read it back, sample UUID `1F3508E0-D277-4395-BF3F-911E95CDE0DA`.
- Authorisation reported: *"Request completed; workout share status: sharing authorized"*.
- The authorisation sheet appeared **on the watch**.
- `isHealthDataAvailable()` returned true on the iPhone (confirmed on the iOS screen) and the watch round trip proves it on the watch.
- Getting to this point took **three separate provisioning obstacles**, none of them HealthKit: an unaccepted Program License Agreement, devices not connected/trusted, then devices not registered to the team. No HealthKit code executed until all three were cleared.

The gap

Four of six predictions confirmed. Two were **not tested** — which is different from being right, and is recorded as untested rather than quietly counted as a win.

#	PREDICTION	OUTCOME
1	HealthKit capability needed on both targets separately	**Confirmed** — set in `project.yml` for both; the generated project carries it twice
2	Missing usage description crashes at authorisation time, not build time	**Confirmed** — tested deliberately, see the sabotage test below
3	`isHealthDataAvailable()` true on both devices	**Confirmed** on both
4	Authorisation sheet appears on the watch	**Confirmed**
5	`requestAuthorization` succeeds even when the user denies; `authorizationStatus` only meaningful for share types	**Confirmed** — tested by deliberate denial, see below
6	The day's time sink is provisioning and signing, not HealthKit	**Strongly confirmed** — three consecutive blockers, all account/provisioning, zero HealthKit

What I actually learned, beyond the score: the mental model that needed correcting wasn't about HealthKit at all. It was that "get an app onto a watch" is a multi-stage negotiation with Apple's account infrastructure — licence agreement, device registration, profile generation — and each stage fails with an error naming a *different* layer than the one you are in. Prediction 6 was the cheapest prediction to make and turned out to be the most useful.

Prediction 5 is worth closing properly before L1 ends: deny a read permission deliberately and confirm the call still reports success. It is thirty seconds and it converts an inherited belief into a tested one.

Amendment — prediction 5, tested by deliberate denial

Method. Turned Heart Rate **off** under Settings → Health → Data Access & Devices → HyroxCoach, leaving Workouts **on**. Relaunched the watch app with the previous instance terminated, then re-ran authorisation and E1.0b.

Result.

OBSERVATION	OUTCOME
`requestAuthorization` after denial	Reported success. No error, no exception, no indication anything had been refused.
Heart rate query	**Zero samples.** No error raised — an empty set, identical in every observable way to "this device has no heart-rate data".
Workouts query	Still returned samples, as expected, since that permission was left on.

Prediction 5 is confirmed. Read denial is genuinely invisible from inside the app: it produces neither an error nor a distinguishable signal, only an empty result.

But a prediction made during this test was WRONG, and it is the more useful finding.

I expected E1.0b's overall verdict to flip to INCONCLUSIVE once heart rate was denied. It stayed **CONFIRMED** — because the probe runs two queries and collapses them into a single aggregate verdict. Workouts still returned data, so the summary reported success while the heart-rate denial sat underneath it, invisible.

So the probe built to catch over-claiming was itself over-claiming. E1.0's original PASS hid the fact that it never tested read access; E1.0b's CONFIRMED hides the fact that one of its two queries returned nothing. Same error, one level up, committed while explicitly trying to avoid it.

Design flaw, stated plainly: an aggregate verdict over multiple independent checks masks any individual failure. A per-type verdict — heart rate CONFIRMED/INCONCLUSIVE, workouts CONFIRMED/INCONCLUSIVE, reported separately — carries the information the aggregate destroys.

Fixed and verified on device, 2026-08-21. `readForeignSamples()` now computes a verdict per type and an OVERALL line that is CONFIRMED only when every type returned data, MIXED when some did not, INCONCLUSIVE when none did. Re-run against the identical condition that fooled the old version — heart rate denied, workouts permitted — it now reports **MIXED**, where before it reported a clean CONFIRMED.

That re-run is the point: the fix was tested against the specific case that exposed the bug, not against a fresh one where it could have passed for the wrong reason. Same discipline the amendments above are about.

Why this matters beyond E1.0. This is the third instance in L1 of a result that reported success while the interesting failure hid inside it. It is the same shape as every row in the failure log: the signal surfaces at one level, the cause sits at another. The lesson for L4 is direct — "the sync worked" will be an aggregate over several things that can each fail independently, and a single green verdict there will hide exactly as much as this one did.

Amendment — prediction 2, tested deliberately

Method. Removed `NSHealthShareUsageDescription` from the watch `Info.plist` with `plutil -remove`, rebuilt, installed to the watch, and tapped authorise. Restored afterwards with `git checkout`.

Result — both halves of the prediction hold.

CLAIM	OUTCOME
The build still succeeds with the key missing	**Confirmed.** `BUILD SUCCEEDED`, no error, **no warning**. The bundle shipped without the key and nothing complained.
Authorisation crashes rather than returning a catchable error	**Confirmed.** Instant termination the moment authorise was tapped. No sheet, no error string, no `catch` block reached.

Why this matters more than it looks. The `requestAuthorization` call sits inside a `do/catch` that logs failures — and that `catch` is useless here. The process is killed before it can run, so no amount of defensive Swift makes a missing plist key recoverable. The only defence is configuration being right in the first place.

The diagnostic signature. A HealthKit app that builds cleanly and dies instantly on the permission tap is almost certainly missing a usage description. Nothing in the build output, the crash timing, or the code points at the plist — it presents as a code bug in the authorisation path, which is where the time gets wasted looking.

This is the same symptom-versus-cause gap that runs through this cycle's failure log: the error surfaces in one layer, the cause sits in another.

Amendment — E1.0b, added after the fact

Why this was added. Reviewing the E1.0 result exposed a flaw in what the PASS actually evidenced. The round trip wrote a sample and read back *the same sample it had just written* — and HealthKit always permits an app to read its own written data regardless of read authorisation. So `PASS: wrote and read back sample UUID …` proved **write access and self-read only**. It was not evidence of read access at all, despite reading like it was.

The probe. `readForeignSamples()` queries heart rate and workouts while excluding this app's own `HKSource`, so any result is data the app did not write. The outcome is deliberately asymmetric and the probe says so in its own output:

- non-empty → **CONFIRMED**, read access is real
- empty → **INCONCLUSIVE**, because HealthKit makes read-denied and no-data-present indistinguishable

Result, 2026-08-20: CONFIRMED. Foreign samples were read successfully, so read authorisation is genuinely granted — consistent with the iPhone's Health → Data Access screen, which showed all four read types enabled.

What this changes about the E1.0 entry. The original PASS stands, but it was over-claimed at the moment it was written: it evidenced less than it appeared to. Read access is now separately evidenced. Prediction 5 remains **half tested** — the deny path is still unexercised, and granting everything is exactly the condition under which the interesting half stays invisible.

The transferable lesson, which is the actual finding here. A test that passes is not automatically evidence of the thing you wanted to test. This one passed for a reason unrelated to the question being asked, and nothing in the green result would have revealed that. Worth carrying into L4, where "the data arrived" will be similarly tempting to read as "the transport is reliable" — when it may only mean the phone happened to be in range.

E1.1 — Baseline: what happens WITHOUT a workout session?

Run: Day 1 (if E1.0 clears) or Day 2

Setup: a watch app with a repeating timer logging via `os_log` every second. No `HKWorkoutSession`. Start it, lower your wrist, wait 3 minutes, raise it.

My prediction

1. The app moves to the background as soon as the wrist drops and the screen turns off.
2. Logging **stops** within a few seconds of backgrounding — likely under 10 seconds, possibly immediately.
3. On raising the wrist, the app resumes, and the timer's elapsed value will be **wrong** if computed by counting ticks, but **right** if computed from a stored start `Date`. This distinction matters more than it looks.
4. There will be no crash and no error. The failure is silent — which is precisely why this is the trap.
5. `os_log` output survives the suspension and is retrievable in Console afterwards, so the gap in the timestamps is itself the evidence.

Actual result

Run 2026-08-21, Apple Watch Ultra 3 (watchOS 26.6). ***Always-On Display: OFF.*** *App launched from the watch itself, no debugger attached.*

READING	VALUE
Ticks recorded	20
Elapsed **by counting ticks**	20.0 s
Elapsed **from stored start `Date`**	97.0 s
Divergence	**77.0 s**
Time from wrist-down to ticks stopping	~8 s

No crash. No error. No callback. The app simply stopped executing and later resumed as though nothing had happened.

FIRST ATTEMPT WAS INVALID — recorded because the reason matters. The initial run was made after installing from Xcode with the debugger still attached, and the ticks never stopped at all. An attached debugger prevents watchOS from suspending the process, so the experiment was measuring the debugger rather than the operating system. Re-run cleanly by stopping the Xcode session and launching from the watch itself.

The gap

All five predictions confirmed, one of them unusually precisely.

#	PREDICTION	OUTCOME
1	App backgrounds as soon as the wrist drops	**Confirmed**
2	Logging stops within a few seconds, likely under 10	**Confirmed** — ~8 s
3	Tick-counted elapsed is wrong; `Date`-computed elapsed is right	**Confirmed** — 20.0 s vs 97.0 s
4	No crash and no error; the failure is silent	**Confirmed**
5	`os_log` survives suspension, so the timestamp gap is the evidence	**Confirmed**

What the number actually means. A tick-counted timer under-reports by exactly the duration the app was suspended. Here it lost 77 of 97 seconds — it saw **21%** of real time. This is not drift or imprecision; it is time the process did not exist for.

Why this decides an architectural rule, not just a coding preference. Every duration in a HYROX session — station times, roxzone transitions, total race time — must be computed from stored timestamps, never by accumulating ticks or incrementing a counter. A wrist-down during a sled push would silently subtract that entire period from the recorded station time, and nothing in the app would report an error. The number would simply be wrong, and plausibly wrong, which is worse.

This is the baseline E1.2 is measured against. E1.2 must be run under identical conditions — Always-On Display OFF, launched from the watch, no debugger — or the comparison means nothing.

E1.2 — The real test: WITH an active `HKWorkoutSession`

Run: Day 2 · **This is the gate.**

Setup: same app, but start an `HKWorkoutSession` + `HKLiveWorkoutBuilder` first. Wrist down, screen off, 15 minutes minimum (a full shortened protocol).

My prediction

1. Logging continues uninterrupted for the entire 15 minutes with the wrist down.

2. Heart rate samples arrive via the live builder without the screen being on.
3. The `activityType` I choose (`.crossTraining` , `.functionalStrengthTraining` or `.highIntensityIntervalTraining`) has **no effect** on whether the session stays alive — it only affects categorisation and energy estimation. HYROX has no dedicated type, so this is an approximation I will have to justify.
4. Battery drain will be noticeable but not prohibitive over 15 minutes.
5. Risk I am least sure about: whether the app is fully *running* or merely *not terminated*. It may be that timers still fire but UI updates are throttled. If so, anything I compute from tick counting is unreliable and I must compute from timestamps instead.

Actual result — GATE PASSED

Run 2026-08-21, ending ~09:44. Apple Watch Ultra 3 (watchOS 26.6). Always-On Display OFF, launched from the watch, no debugger. Same conditions as E1.1.

Session state throughout: `Workout running` / `Workout collection running` . Heart rate **84 bpm**, arriving live with the screen off.

Two readings were taken, which turned out to matter more than one.

	READING A (09:43)	READING B (09:51, FINAL)
Ticks recorded	1,134	1,610
Elapsed **by counting ticks**	1,134.0 s	1,610.0 s
Elapsed **from stored `Date`**	1,149.0 s	1,630.0 s
Divergence	**15.0 s**	**20.0 s**
Loss as a proportion	1.31 %	1.23 %
Duration	~19 min	~27 min

Between the two readings, 481 s of real time passed and 476 ticks were recorded — 5 s lost in that window, about 1.04 %.

Direct comparison against the E1.1 baseline, run under identical conditions:

	NO SESSION (E1.1)	WORKOUT SESSION (E1.2, FINAL)
Real elapsed	97.0 s	1,630.0 s
Counted by ticks	20.0 s	1,610.0 s
Time lost	77.0 s	20.0 s
Proportion lost	**79.4 %**	**1.23 %**

The gap

#	PREDICTION	OUTCOME
1	Logging continues *uninterrupted* for the full duration	**Mostly confirmed, and usefully wrong in detail.** It continued, but not uninterrupted — 15 seconds of ticks were still lost.
2	Heart rate arrives via the live builder without the screen on	**Confirmed** — 84 bpm with the screen off
3	`activityType` does not affect whether the session stays alive	**Untested** — only `.crossTraining` was used
4	Battery drain noticeable but not prohibitive	**Untested** — not measured
5	Least confident: the app may be *not terminated* rather than *fully running*, with timers throttled	**Confirmed** — this is exactly what the 15 s shortfall shows

The headline answer to L1's question. An `HKWorkoutSession` is what keeps the app executing while the wrist is down and the screen is off. Without it the app lost 79% of elapsed time; with it, 1.3%. That is the difference between an app that records a workout and one that merely appears to.

The more interesting result is that 15 seconds, not the pass. Prediction 5 — the one flagged as least confident — was right. A workout session prevents *suspension*; it does not guarantee a timer fires every second. The process stays alive while individual ticks are still coalesced or dropped.

So the architectural rule from E1.1 is not softened by this result, it is **reinforced**: durations must come from stored timestamps even *with* an active workout session.

The second reading is what makes this conclusive. A single measurement could have been dismissed as a start-up artifact — some cost paid once while the session spun up, then negligible. It is not. Loss held at 1.31 % after 19 minutes and 1.23 % after 27, with 1.04 % across the window between them. **The drift is steady and proportional to elapsed time, not a fixed one-off cost.**

That converts an observation into an extrapolation. At ~1.2 %, a full HYROX race of roughly 90 minutes would lose about **65 seconds** to tick-counting — comfortably enough to misreport a station, a transition, and the total. On a 4-minute sled push it is a ~3-second error. Silent, plausible, and undetectable from inside the app.

Which is the same shape as everything else in this cycle. "Workout running" is a green signal that means the session is alive. It does not mean every tick fired. Believing the stronger claim because the weaker one is true is precisely the error the rest of L1 documented.

E1.3 — Does `recoverActiveWorkoutSession` do what the docs say?

Run: Day 2

Setup: start a session, force-quit the watch app, relaunch, call `HKHealthStore.recoverActiveWorkoutSession(completion:)`.

My prediction

1. It returns the still-running session rather than nil, and `session.associatedWorkoutBuilder()` gets me back to the builder with samples collected during the gap intact.
2. HealthKit's half survives. **My half does not** — current station, segment times and logged transitions are gone unless I persisted them, which is the whole reason L2 exists.
3. The recovered session's `startDate` is the original start, not the relaunch time.
4. Lowest-confidence prediction in this cycle. Recovery semantics have shifted across watchOS releases and I have not verified this on Stanley's version. If it behaves differently, L2's design changes and that is worth knowing on Day 2 rather than Day 7.

Actual result — RECOVERED SUCCESSFULLY (second attempt)

Run 2026-08-21, 10:21. Apple Watch Ultra 3 (watchOS 26.6). Session started 10:19:35, force-quit via Side button + Digital Crown, relaunched, recovery attempted as the first action.

```
Recovered state running; start 21 Aug 2026 at 10.19.35;
associated builder: yes
```

READING	VALUE
Session returned	Yes, state `running`
`associatedWorkoutBuilder()`	Yes
Recovered `startDate`	**10:19:35** — the original start
Time of recovery	~10:21, roughly 90 s later

FIRST ATTEMPT FAILED — and the reason is itself a design constraint.

At 10:14 recovery threw:

```
Task server endpoint for '43DFBD17-…-14D9DA5FA508' already exists
(for instance 'E8AB3AF1-F0AC-4632-B29E-9D72CC693B96')
```

Cause: the E1.2 workout was **still running** and its session manager still held a live `HKWorkoutSession`. Recovery was called while an instance already existed, and the framework correctly refused to issue a second one. The test was malformed, not the API.

The gap

#	PREDICTION	OUTCOME
1	Returns the still-running session; `associatedWorkoutBuilder()` gets back to the builder	**Confirmed** — state `running`, builder present
2	HealthKit's half survives; my semantic half does not	**First half confirmed. Second half not yet testable** — there is no semantic state to lose, because no state machine exists until L2. Recorded as untested rather than assumed.
3	The recovered `startDate` is the original start, not the relaunch time	**Confirmed** — 10:19:35, roughly 90 s before recovery
4	Lowest-confidence prediction; recovery semantics may differ on this watchOS version	**Resolved.** Recovery behaves as documented *when called correctly*. The uncertainty was warranted, but the risk turned out to sit in the calling sequence rather than the API.

The real finding is the constraint the failed attempt exposed. Recovery cannot be called while the process already holds a session object for that workout. It must be **the first thing a relaunched app does**, before any screen, view model or manager instantiates a session. Touch E1.2's screen first and you re-register an endpoint and reproduce the failure — for the correct reason, which is what makes it dangerous.

Direct consequence for L2. Recovery is not a repair step you reach for after noticing something is wrong; it is a launch-path decision made before normal startup runs. Combined with prediction 2 — HealthKit restores its own record while the semantic layer does not — L2's design follows: persist semantic state on every transition, and on launch attempt recovery *first*, then rehydrate the semantic layer alongside whatever HealthKit hands back.

Prediction 3 matters more than it looks. Given that E1.1 and E1.2 established every duration must be computed from stored timestamps rather than counters, the recovered `startDate` is the field all of those computations hang from. It surviving a force-quit intact is what makes recovery useful rather than merely possible.

Amendment — E1.3 is reproducible, and the earlier conclusion was too simple

Reported behaviour, reproducible across attempts:

SEQUENCE	RESULT
Start workout → force-quit → relaunch → attempt recovery	**"Recovery failed"** – endpoint already exists
Then end the workout → attempt recovery	**"No active workout session was available to recover"**
Then attempt recovery **again**	**"Recovered state running"**

The probe code was audited and is not the cause. `RecoveryProbe.attemptRecovery()` sets `latestResult` on every branch – success, nil, and error – and each tap issues a fresh `recoverActiveWorkoutSession` call. There is no cached or stale result. The three outcomes are the framework's, not the app's.

What this establishes (recorded fact):

1. Recovery immediately after a force-quit **reliably fails** while a workout is active, with an endpoint-already-exists error.
2. After the workout is ended, recovery reports **no session available**.
3. A subsequent attempt then returns a **running session with the original `startDate`**.

What this suggested at the time (hypothesis – since FALSIFIED by E2.0 on 2026-08-24): an app holding an active `HKWorkoutSession` is not really terminated by a force-quit – the system keeps or relaunches an instance to sustain the session, so its endpoint is still registered when the user reopens the app. **E2.0 showed the app stays dead. This explanation is wrong. The cause was later identified by E2.0b: the conflict occurs whenever the process already holds a live `HKWorkoutSession` object – in E1.3's case, E1.2's still-running workout in the same process.** Ending the workout releases the endpoint held by *this* process's session object, after which recovery can return the session that is still running at system level. That would explain all three states and the preserved start time, but the mechanism has **not** been proven here.

How this changes the earlier conclusion. The original entry recorded recovery as working, with the failed first attempt written off as a malformed test. That was true but incomplete. The failure is not a one-off procedural slip – it is the **reliable** outcome of the exact scenario L2 must handle: the app dying mid-workout. Recovery succeeded only after an intervening end-workout, which is not something a crashed app gets to do.

Consequence for L2, and it is a significant one. "Force-quit the app and recover the session" cannot be assumed to work as a straight sequence. Before designing persist-on-transition, L2 must first establish what genuinely happens to a watch app that dies mid-workout – whether the system resurrects it, and what state the process comes back in. The recovery path may need to tolerate an endpoint conflict on launch and retry, rather than calling recovery once and trusting the result.

Status: recovery is confirmed possible, but the launch-path design is now an open question for L2 rather than a settled one.

CYCLE L1 · RESULTS SUMMARY

Status: COMPLETE. Gate passed on Day 2 (2026-08-21), on schedule.

Conditions for every measurement below – Apple Watch Ultra 3 (watchOS 26.6), iPhone 16 Pro Max (iOS 26.6.1), Xcode 26.6. Always-On Display **OFF**, app launched **from the watch**, **no debugger attached**. E1.1 and E1.2 are directly comparable because both were run this way; any result taken under different conditions is marked where it appears.

The question, and the answer

> `Why does a watch app stop recording, and what does HKWorkoutSession actually change?`

Answer. Without a workout session, watchOS suspends the app within about 8 seconds of the wrist dropping, and it loses 79% of elapsed time while reporting no error of any kind. With an active `HKWorkoutSession` the app keeps executing and loses about 1.2%. The session is what buys background execution – but it buys *survival of the process*, not *reliable execution of every timer*.

Results by experiment

EXPERIMENT	QUESTION	RESULT
E1.0	Can the app reach HealthKit on real hardware?	**PASS** – authorised on the watch; wrote 1 kcal and read it back, sample `1F3508E0-D277-4395-BF3F-911E95CDE0DA`
E1.0b	Was *read* access actually granted?	**CONFIRMED** – foreign-source samples read. Added because E1.0's pass could not evidence read access
P2 (sabotage)	Does a missing usage description fail the build or crash at runtime?	**Crash.** Build succeeded with no error or warning; instant termination on tapping authorise, before any `catch`
P5 (denial)	Is a denied read permission detectable from inside the app?	**No.** Authorisation still reported success; the query returned an empty set with no error
E1.1	What happens with **no** workout session?	Ticks stopped ~8 s after wrist-down. **20 ticks / 20.0 s counted vs 97.0 s real – 77.0 s lost (79.4%)**
E1.2	What happens **with** an active session?	**GATE PASSED.** Two readings: 1,134 ticks / 1,149.0 s (15.0 s lost, 1.31%) and 1,610 ticks / 1,630.0 s (20.0 s lost, 1.23%). Heart rate 84 bpm live with the screen off
E1.3	Does `recoverActiveWorkoutSession` work?	**Possible, but the launch path is an OPEN QUESTION.** Recovery after force-quit *reliably fails* while a workout is active. Success required an intervening end-workout, which a crashed app cannot perform

The headline measurement

	NO SESSION (E1.1)	WORKOUT SESSION (E1.2, FINAL)
Real elapsed	97.0 s	1,630.0 s
Counted by ticks	20.0 s	1,610.0 s
Time lost	77.0 s	20.0 s
Proportion lost	**79.4 %**	**1.23 %**

Two E1.2 readings taken 481 s apart lost 5.0 s between them (1.04%), so the shortfall is **proportional to elapsed time, not a one-off start-up cost**. Extrapolated, a ~90-minute HYROX race would lose roughly **65 seconds** to tick-counting.

What these percentages do and do not mean — checked in code 2026-09-04

They measure how often the app was allowed to run. They do not measure the accuracy of any recorded time. Stated carelessly, "the app loses 1.23%" reads as though the race record drifts. It does not. The two numbers describe execution frequency, and nothing else.

Every time value in the product path is a timestamp subtraction, not a count:

```
let elapsed = Date().timeIntervalSince(state.stateStartedAt)
let duration = segment.endedAt.timeIntervalSince(segment.startedAt)
```

`ProtocolMachine` contains no tick counting anywhere. **The recorded race has zero time dilation and never had any.**

The 1.23% belongs to the instrument, not the product. `BackgroundProbe` and `WorkoutSessionManager` each publish two figures deliberately, side by side:

	HOW IT IS PRODUCED	DILATES?
`elapsedByTicks`	Counts timer firings	**Yes** – watchOS coalesces timers
`elapsedByDate`	Subtracts two timestamps	**No**

E1.2 existed to compare them. The gap between them *is* the finding: it is how the survival of the process was measured. Reading it as an error in the race record inverts what the experiment did.

What dilation genuinely remains: display refresh, and it is cosmetic. When watchOS coalesces timers the on-screen number updates every two or three seconds instead of every second. The value is correct whenever it refreshes; it simply refreshes late, and looks frozen.

This is removable for one line, and belongs in the lo-fi design:

```
Text(timerInterval: startDate...Date.distantFuture)
```

The system renders that clock, not the app. It continues smoothly while the app is not executing at all – no timer, no coalescing, no visible stutter.

Conclusion. Recorded times already carry no dilation. Displayed times need not carry any either.

Confirmed on device 2026-09-04. `Text(timerInterval:)` was added to the watch app as a fourth readout beside the tick counter, and a workout was run with a deliberate wrist-down interval. Observed: **`By ticks` fell behind; `By date` and `By system` both stayed correct.**

This is a qualitative observation — the three figures were read off the wrist, not captured to the log — so it is recorded as direction confirmed, not as a measured gap. The measured figures remain the ones from E1.1 and E1.2.

What it demonstrates. The system-rendered clock keeps time correctly across an interval in which the app's own timer demonstrably did not fire. It therefore needs no execution to stay accurate, which is exactly the property the display requires.

It also makes L1 visible in one screen. The tick counter drifting away from two correct clocks, live on the wrist, shows the finding more directly than the percentages do. Kept in the app for that reason as well as for the display fix.

Prediction tally

EXPERIMENT	CONFIRMED	WRONG	UNTESTED
E1.0 (6 predictions)	6	0	0
E1.0b + side-prediction	1	**1**	0
E1.1 (5 predictions)	5	0	0
E1.2 (5 predictions)	2 confirmed, 1 refined	0	2
E1.3 (4 predictions)	2, 1 resolved	0	1

Untested and honestly marked, not counted as wins:

- E1.2 · whether `activityType` affects staying alive — only `.crossTraining` was used
- E1.2 · battery drain — never measured
- E1.3 · whether semantic state is lost on recovery — **untestable until L2 exists**, since there is no semantic state to lose yet

The one wrong prediction was the most useful. Expecting E1.0b to report INCONCLUSIVE after denying heart rate exposed that the probe aggregated two independent queries into a single verdict, hiding an individual failure. Fixed to report per-type verdicts, and re-verified against the same case.

Architectural rules this cycle established

These are the durable output of L1 — decisions now backed by measurement rather than assumption.

1. **Every duration must be computed from stored timestamps, never from accumulated ticks or counters.** Holds even with an active workout session, where tick-counting is still wrong by ~1.2%. On a 4-minute sled push that is a ~3-second silent error.
2. **An `HKWorkoutSession` must be running for any live capture.** Without it there is no recording, and no error to tell you so.
3. **Configuration correctness is not defensible in code.** A missing Info.plist usage description kills the process before any error handler runs.
4. **Read permission state cannot be inferred from inside the app.** An empty result is indistinguishable from a denial. Anything built on "we got no data" must treat that as ambiguous.
5. **Verification must be per-check, never aggregated.** A single verdict over independent checks hides individual failures.
6. **The recovered `startDate` survives and is the anchor for every derived duration** — which is what makes recovery worth having at all.

The pattern that ran through the whole cycle

Six times, a success signal at one level concealed a failure at another:

GREEN SIGNAL	WHAT IT HID
Round trip `PASS`	Read access was never tested
`BUILD SUCCEEDED`	A fatal missing configuration key
Authorisation `success`	Read access had been denied
Probe `CONFIRMED`	One of its two checks returned nothing
Install exit code `0`	The install had actually failed
E1.1 ticks continuing	A debugger was holding the app alive

A seventh belongs to the record-keeping rather than the tooling: E1.3's first failure was written up as a one-off malformed test, and only proved reproducible because the sequence was repeated rather than accepted.

This was not L1's subject. It is L1's most transferable finding, and the direct warning for L4, where "the sync worked" will be an aggregate over transfer, delivery, duplicate handling and deduplication — each able to fail independently, with no sample identifier on screen to make it obvious.

Open questions carried into L2

1. **What actually happens to a watch app that dies mid-workout?** Does the system resurrect it to sustain the session? What state does the process return in? This must be answered before persistence is designed, not after.
2. **Can recovery be called reliably on launch**, or must the launch path tolerate an endpoint conflict and retry?
3. Does `activityType` affect session longevity? (Untested; low priority.)
4. What is the battery cost of a long session? (Untested; matters for a 90-minute race.)

CYCLE L2 · Recoverability — Days 3–4 (Aug 24–25)

Learning question: What has to be true for a workout to survive the app dying?

Opened 2026-08-21. All predictions below were written before any L2 code was created.

L1 changed this cycle's shape. The plan was to design persist-on-transition and call recovery on launch. E1.3 showed recovery *reliably fails* in the crash-like case, so a prior question has to be answered first: what actually happens to a watch app that dies mid-workout? E2.0 exists to answer that, and everything after it depends on the answer.

E2.0 — What actually happens when the app dies mid-workout?

This is the question E1.3 left open. It must be answered before persistence is designed.

Setup: start a workout session, kill the app several ways — force-quit gesture, and a deliberate crash (`fatalError`) — then observe. Does the process come back? When? In what state? Is an endpoint already registered when the user reopens it?

My prediction

1. **Force-quit does not really kill it.** An app holding an active `HKWorkoutSession` is relaunched by the system in the background to sustain the session. This is the hypothesis that explains E1.3's three-state sequence, and it is the single most important thing to confirm or kill.
2. Because of #1, by the time the user reopens the app, an instance already exists and its endpoint is registered — which is why recovery reports "already exists" rather than handing the session back.
3. **A deliberate crash may behave differently from a force-quit.** A force-quit is a user gesture the system may treat as "keep the workout"; a crash is a fault. If they differ, the crash case is the one that matters, since it is what actually happens in the field.
4. `recoverActiveWorkoutSession` will therefore need to tolerate the conflict — either by retrying, or by detecting that the process already holds a session and using that instead of recovering.
5. **Lowest confidence in this cycle.** All of the above is inference from one reproducible symptom, not from documentation. I expect at least one of these to be wrong.

Actual result — PREDICTION 1 FALSIFIED

*Run 2026-08-24, Apple Watch Ultra 3. App installed with **no debugger attached**, launched from the watch. **A workout session was confirmed running at the moment of the crash.***

READING	VALUE
Launch 1 (opened by hand)	08:57:22
Deliberate crash	08:58:14 — 52 s into the session
Launch 2	09:00:15
First launch after crash	**2 m 0 s**
Total launches	**2**
Latest launch followed a crash	YES

The system did NOT relaunch the app. Two launches, both performed by hand — the first to start, the second after deliberately waiting two minutes. The gap of exactly 2 m 0 s matches the wait, not an autonomous restart. Had watchOS resurrected the process to sustain the workout, a third launch would have appeared within seconds of the crash. None did.

The gap

#	PREDICTION	OUTCOME
1	Force-quit/crash does not really kill it; the system relaunches the app to sustain the session	**WRONG.** The app stayed dead.
2	Therefore an instance already exists on reopening, which is why recovery reports "already exists"	**WRONG – it depended on #1.**
3	A deliberate crash may behave differently from a force-quit	**Untested** – force-quit comparison not yet run
4	Recovery will need to tolerate the conflict	**Undecided** – the conflict's cause is now unknown
5	Lowest confidence in this cycle; I expect at least one of these to be wrong	**Correct.** Two were wrong, including the central one.

E1.3 is now MORE mysterious, not less. The background-relaunch hypothesis was the only explanation on the table for "task server endpoint already exists", and it is dead. If no process survives the crash, nothing should hold that endpoint when the app is reopened – yet E1.3 reproduced the conflict every time.

New hypothesis, and the obvious next test: `recoverActiveWorkoutSession` may not be idempotent. E1.3's own sequence is the clue – two identical consecutive calls returned *different* answers ("no active session", then "recovered state running"). A pure query cannot do that. It suggests the call itself has side effects: registering an endpoint, or transitioning state, so that the second call observes a world the first one changed.

Note also that `RecoveryProbe` **discards the recovered session** – it calls `associatedWorkoutBuilder()` and keeps nothing. If a successful recovery registers an endpoint that is only released when the session object is retained and properly ended, a discarded session could leave a dangling registration behind.

Consequence for L2. The cycle cannot proceed to persistence design yet. E2.2 assumed the app dies and is restored; E2.0 confirms the app really does die, which is good – but the recovery path is now less understood than when the cycle opened. **E2.0b must come next:** call recovery twice in a row on a clean launch and observe whether the call changes what it reports.

Worth stating plainly: this cycle's central prediction was wrong, and I would not have found that out by building persistence on top of it. It would have surfaced somewhere around Day 8, as a bug that looked like something else.

E2.0b — Is `recoverActiveWorkoutSession` idempotent?

Added 2026-08-24 after E2.0 falsified the background-relaunch hypothesis. Predictions written before the code was changed.

Why this exists. E1.3 showed two identical consecutive calls returning different answers – "no active session", then "recovered state running". A pure query cannot do that. Either the call has side effects, or something outside the app changes between calls. E2.0 ruled out a resurrected process, so the cause must be inside the call or inside HealthKit's own bookkeeping.

A supporting detail in my own code: `RecoveryProbe` currently **discards** the recovered session. It calls `associatedWorkoutBuilder()` and retains nothing. If a recovered session registers an endpoint that is only released when the object is retained and properly ended, discarding it could leave a registration behind that nothing can clear.

Setup: on a clean launch with a workout running, call `recoverActiveWorkoutSession` **twice in succession**, reporting both results separately. Then repeat with the returned session **retained** in a property rather than discarded, and compare.

My prediction

1. **The two calls will not agree.** If they return the same result twice, the non-idempotence hypothesis dies immediately and E1.3's behaviour needs a different explanation again.
2. **Retaining the session will change the outcome.** Specifically, I expect discarding it to be what leaves the dangling endpoint, so the retained version should either succeed cleanly or fail *consistently* rather than alternating.
3. If the first call fails with "already exists" on a genuinely clean launch – no prior recovery in that process – then the registration survives process death, which the app cannot see or control. That would make it a HealthKit-level constraint rather than an app bug.
4. **The honest possibility I cannot rule out:** the mechanism may be none of the above. Two hypotheses have already died this cycle, and I have no documentation for any of this – only observed behaviour.
5. Whatever the mechanism, the practical rule for L2 will be the same: recovery must be attempted defensively, its result checked rather than assumed, and the returned session retained.

Actual result – NOT IDEMPOTENT. CONFIRMED, and it explains E1.3.

Run 2026-08-24, 09:30. Clean install, no debugger, workout started then force-quit, recovery tapped as the first action on relaunch.

```
CALL 1: recovered (state: running; startDate: 24 Aug 2026 at 9.29.49)
CALL 2: error: Task server endpoint for
        'AE606960-1713-4A2E-A074-4A75B063496A' already exists
        (for instance '472919D3-D983-4B57-8E5F-C465356092B0')
VERDICT: DIFFERENT
```

Two identical consecutive calls. The first succeeded. The second failed with the exact error that has haunted this cycle since E1.3.

The gap

#	PREDICTION	OUTCOME
1	The two calls will not agree	**Confirmed** – VERDICT: DIFFERENT
2	Retaining the session will change the outcome; expect consistent behaviour rather than alternating	**Confirmed in effect, wrong in mechanism.** Retention did not prevent the conflict – the retained session *is* the conflict. But behaviour is now deterministic rather than alternating, which is what mattered.
3	If call 1 fails on a genuinely clean launch, the registration survives process death	**Antecedent did not occur.** Call 1 *succeeded* on a clean launch, so a registration does **not** survive process death. This rules out the HealthKit-level-constraint scenario.
4	The mechanism may be none of the above	**Not needed.** It was one of the above.
5	Whatever the mechanism, recovery must be defensive, checked, and the session retained	**Confirmed as the right rule**, now for a known reason rather than a precautionary one.

The mechanism, finally

`recoverActiveWorkoutSession` fails with "task server endpoint already exists" whenever **the process already holds a live `HKWorkoutSession` object for that workout** – regardless of where that object came from.

This explains every observation in the cycle:

OBSERVATION	EXPLANATION
E1.3, first attempt failed	E1.2's workout was still running **in the same process**, so `WorkoutSessionManager` held a live session object and its endpoint was registered
E1.3, after ending the workout: "no active session"	The in-process session had been ended, releasing its endpoint
E1.3, next tap: "recovered state running"	With the endpoint free, recovery could return the session still running at system level
E2.0: no background relaunch	Correct and irrelevant – the conflict was never about a surviving process
E2.0b CALL 1 succeeded	Clean launch, nothing held a session
E2.0b CALL 2 failed	CALL 1's own retained session now holds the endpoint

The earlier "malformed test" reading of E1.3 was right after all, but for a reason nobody had identified at the time. It was then wrongly overturned in favour of a background-relaunch hypothesis that E2.0 killed. Two wrong explanations preceded the correct one, and both were written down before being disproved – which is the point of keeping them.

L2's open question is now closed: the launch path

Recovery is **not** a repair step to be reached for when something looks wrong. It is a **one-shot decision made at launch, before anything else can create a session**:

1. On launch, call `recoverActiveWorkoutSession` **exactly once**, before any view, view model or manager can instantiate an `HKWorkoutSession`.
2. **Retain** whatever it returns and treat it as *the* session for the process.
3. **Never call recovery again** in that process – a second call is guaranteed to fail against the first's own registration.
4. If recovery returns nil, only then create a new session.
5. If recovery throws "already exists", the process has a bug: something created a session before recovery ran.

Point 5 is the useful one. That error is not an environmental hazard to be retried around – it is a **self-inflicted ordering error**, and it should be treated as a programming mistake rather than a condition to tolerate.

E2.1 — The state machine, on paper before in code

Setup: draw the HYROX session as typed states and transitions. Deliberately a design artifact, not code. Kept in two versions – before implementation and after – because the diff is the learning.

My prediction

1. The happy path is small: `idle → run(leg n) → transition → station(n) → transition → run(leg n+1) → … → complete`. Roughly five state *kinds*, repeated sixteen times.
2. **The awkward paths will outnumber the happy path.** Pause, resume, interruption by a call, app death, accidental advance, ending mid-station. I expect the recovery and correction logic to be larger than the logic that runs when nothing goes wrong.
3. **Accidental advance will be the hardest to model.** Undoing a mis-tap means restoring the previous state *and* its timestamps, so the machine needs history, not just a current state.

4. Every state must carry a **start timestamp**, never an accumulating duration — a direct consequence of L1's measured rule.

Actual result — v1 written 2026-08-24

Design document at `design/L2-state-machine.md`. 183 lines, written before any implementation, with a v2 to follow after coding so the diff can be examined.

MEASURE	ORDINARY PROGRESSION	AWKWARD PATHS ONLY	TOTAL
State kinds	6 — Idle, Preparing, Running, InRoxzone, InStation, Completed	4 — Recovering, Paused, Interrupted, Abandoned	10
Transition rules	6	**20**	26
Ratio (awkward : ordinary)		states **0.67 : 1** · transitions **3.33 : 1**	

The gap

#	PREDICTION	OUTCOME
1	Happy path is small — roughly five state kinds, repeated sixteen times	**Confirmed** — six ordinary state kinds
2	The awkward paths will outnumber the happy path	**MIXED, and the split is the finding.** By transition rules, overwhelmingly yes: 20 against 6, more than three to one. By state kinds, no: 4 against 6.
3	Accidental advance will be the hardest to model; undo needs history, not just current state	**Confirmed** — it is the only path requiring a bounded LIFO `undoHistory` of complete state snapshots, including original timestamps, stored inside the same atomic envelope
4	Every state must carry a start timestamp, never an accumulating duration	**Confirmed and held throughout** — a direct consequence of L1's measurement

What the mixed result actually taught. The prediction assumed awkward handling would show up as *more states*. It does not. It shows up as **more edges between the same states** — pause, interruption, death and mis-tap are mostly transitions into and out of a small number of extra states, not a proliferation of new ones.

That distinction matters for implementation. A design measured by state count looks deceptively simple; the complexity lives in the transition table, which is where the bugs will be too. Had the count been reported only by states, the prediction would have read as falsified and the real conclusion — that correction logic dominates — would have been missed.

Prediction 3 produced the most concrete design consequence. Undo cannot be reconstructed from the current state: restoring a mis-tap means restoring the *original* timestamps, not creating a fresh state that looks similar. v1 bounds the history to the two most recent user-driven advances, which covers an immediate mis-tap and one compound correction without turning persistence into an unbounded event log.

Nine open questions are recorded at the end of the design, several of which are experiments rather than decisions — notably whether temp-file-then-rename always leaves a complete envelope, and whether station boundaries can round-trip as offsets from the recovered `startDate` without drift. Those two feed directly into E2.2 and E2.3.

E2.1 verification pass — 2026-08-24

A design cannot be run, so it was audited instead: the tables were parsed and counted independently of the document's own claims, rather than taking its summary on trust.

CHECK	METHOD	RESULT
Transition count	Parsed rows from the table	**26** — matches the claim
Ordinary vs awkward split	Counted the class column	**O = 6, A = 20** — matches the claimed 3.33 : 1
State count	Parsed the states table	**10** — matches
Unreachable states	Every state checked for an incoming transition	**None.** Every state is reachable
Dead-end states	Every state checked for an outgoing transition	**Only `Abandoned`** — correct, it is terminal
Every state carries a start timestamp	Field column checked per state	**All 10 do.** No state stores a duration

One structural detail worth noting: `Completed` is *not* a dead end. It has an outgoing transition, because undo from `Completed` is reachable. That is deliberate and matches the mis-tap handling — but it means the machine has a terminal state that can be re-entered, which is the kind of thing that produces surprises in implementation. Flagged rather than treated as a defect.

A gap was reported here and was WRONG. Retained as evidence rather than deleted.

The audit claimed the design gave the call-recovery-once rule without its measured reason. It does not. The design already states, in the launch-time recovery section: *"This strict order is measured, not defensive folklore: the API is not idempotent, and a second call fails against the first call's own session… It is a programming-ordering bug and must not be retried around."*

Cause of the false finding: the audit grep used `\|` for alternation while running with `-E`, where alternation is `|`. The pattern was matched literally, returned zero, and the zero was read as "absent" without checking the surrounding text.

This is the project's own recurring failure, committed by the auditor. A tool returned a clean-looking result — a count of 0 — and it was trusted instead of verified. Every other instance in this workbook involved a green signal hiding a failure; this one is the inverse, a null result manufacturing one. The lesson is the same: **a tool's output is evidence about the tool as much as about the subject.**

Practical consequence: a negative grep result proves nothing until the pattern itself has been shown to match something it should.

Verdict: the design's claims about itself are accurate. The counts are real, the graph is well-formed, and the timestamp constraint holds throughout. The recorded E2.1 result stands unchanged.

E2.2 — Persist on every transition, then kill it

Setup: write semantic state to disk at every transition. Kill the app mid-station. Relaunch. Compare what was restored against what was true at the moment of death.

My prediction

1. Writing on every transition is cheap. A HYROX race has ~17 transitions in ~90 minutes, not one per second, so cost is irrelevant and there is no reason to batch.
2. **Nothing meaningful is lost, provided timestamps are persisted rather than durations.** If the file records "station 3 started at 10:19:35", elapsed time is recomputable at any later moment. If it records "station 3 has run for 47 seconds", everything after the last write is lost. This is L1's rule applied to storage.
3. **The write must be atomic.** A crash during a write corrupts the file, and a corrupt file is worse than a missing one, because a missing file is obviously missing. Write to a temp file and rename.
4. What *is* genuinely lost: any transition that happened between the last successful write and the death. With per-transition writes that window is one transition at most.
5. I expect the first implementation to fail on relaunch for a boring reason — a decoding error or a missing file on first run — rather than anything conceptually interesting.

Actual result — NOTHING LOST

*Run 2026-08-24. App killed by force-quit while in **Run 2** (not mid-station, despite the shorthand). No debugger.*

Before the kill, 10:02

```
No persisted state to restore.          ← this launch started fresh
Current: Run 2
Current elapsed: 21.4s
Completed segments: 4
  1. Preparing:           4.0s
  2. Run 1:               1.8s
  3. Roxzone 1:           5.5s
  4. Station 1 — SkiErg:  5.8s
```

After force-quit and relaunch, 10:03

```
Restored Run 2; state began 36.0s ago; 4 segments complete.
Current: Run 2
Current elapsed: 59.0s
Completed segments: 4
  1. Preparing:           4.0s
  2. Run 1:               1.8s
  3. Roxzone 1:           5.5s
  4. Station 1 — SkiErg:  5.8s
```

CHECK	RESULT
State kind restored	Run 2 — correct
Completed segments	4, with **identical durations** before and after
Elapsed across the death	21.4 s → **36.0 s at restore** → 59.0 s — the dead time is included
Absent-state path	The earlier launch correctly reported "No persisted state to restore" rather than inventing one

The elapsed figures are the proof. The current state's clock did not restart, pause, or lose the interval during which the process did not exist. Elapsed was recomputed from the stored `stateStartedAt` at the moment of display, so the app was able to report time it was never running for.

The gap

#	PREDICTION	OUTCOME
1	Writing on every transition is cheap; no reason to batch	**Not measured.** No lag was observable, but nothing was timed. Recorded as unmeasured rather than confirmed.
2	Nothing meaningful is lost, provided timestamps are persisted rather than durations	**Confirmed, decisively.** Segment durations identical; current elapsed carried straight through the death.
3	The write must be atomic — a crash during a write corrupts the file	**Now confirmed by E2.2b.** Tested at three injection points including a complete-but-uncommitted temp file; no partial state ever observed.
4	At most one transition is lost — whatever happened between the last write and the death	**Untested.** The kill landed mid-state, not mid-transition, so the lossy window was never exercised.
5	The first implementation will fail on relaunch for a boring reason — a decode error or missing file	**WRONG.** It worked first time.

Prediction 2 closes the loop opened in L1. The measured rule was "compute durations from timestamps, never from counting" because a suspended app loses time silently. E2.2 shows the same rule surviving a stronger test: not merely a suspended process, but one that *ceased to exist*. A stored timestamp does not care that the app was dead; a stored duration would have lost every second of it.

Three of five predictions remain untested, and that is the honest state. Atomicity, the lossy transition window, and write cost were all designed for but not exercised. The experiment proved that persistence *works*; it did not prove that persistence is *safe under adversarial timing*. Those are different claims, and only the first has evidence.

The one wrong prediction is worth keeping. I expected a boring first-run failure — a decode error, a missing file. There wasn't one. Predicting friction is cheap and it did not happen here; the interesting failures in this project have consistently come from somewhere other than where they were expected.

E2.2b — Is the atomic write actually atomic?

Added 2026-08-24. E2.2 proved persistence works; it did not prove it survives a kill landing inside a write. Predictions written before the code was changed.

Why a deterministic test rather than a lucky one. Force-quitting and hoping to land inside a write is not an experiment, it is a raffle — the write window is milliseconds wide. Instead the save path gets **injectable crash points**, so the dangerous moment can be hit on purpose:

- `beforeTempWrite` — crash before anything is written
- `duringTempWrite` — write a deliberately truncated temp file, then crash
- `afterTempWriteBeforeReplace` — complete temp file exists, crash before the atomic replace

After each, relaunch and ask what `load()` returns.

My prediction

1. **All three crash points leave the previous complete envelope intact.** `load()` returns the last successfully committed state — never a partial one, never a throw. This is the whole claim behind temp-file-then-replace.
2. **`afterTempWriteBeforeReplace` is the real test.** A complete, newer temp file exists alongside the older real file. If the implementation ever reads the temp file, or replaces on read, this is where it breaks. I expect it not to.
3. **Stale `.tmp` files accumulate** and nothing cleans them up. Harmless for correctness, but a small leak, and worth knowing before it becomes a mystery later.
4. **The state lost is the in-flight transition only** — the app returns to the state before the transition that was being written. That is the acceptable failure mode E2.2 predicted but never exercised.
5. **Lowest confidence: whether `replaceItemAt` is genuinely atomic on watchOS.** It is documented as such, but this project has already found documented behaviour that did not hold. If it is not atomic, prediction 1 fails and the persistence design needs a different mechanism — which would be far better to learn now than in L4.

Actual result — ATOMIC. All three crash points survived.

Run 2026-08-24, 10:25–10:32. Deterministic crash injection inside the save path, no debugger, force-quit not involved.

ROUND	CRASH POINT	REAL FILE	TEMP FILE	RESTORED
1	`beforeTempWrite`	**valid**, 1869 B	absent	Station 1, 3 segments — the pre-transition state
2	`duringTempWrite`	**valid**, 2554 B	present, **1619 B** (truncated)	Run 2, 4 segments — pre-transition
3	`afterTempWriteBeforeReplace`	**valid**, 2552 B	present, **3239 B** (complete)	Run 2, 4 segments — pre-transition

Round 3 is the decisive one and it passed. A complete, newer temp file of 3239 bytes sat on disk beside a 2552-byte committed file — larger, more recent, and entirely legitimate-looking. The app ignored it and loaded the committed state. Had the implementation ever preferred the temp file or replaced on read, this is the only round that would have caught it.

An arithmetic cross-check the experiment produced by accident. Round 2's truncated temp was 1619 bytes; round 3's complete temp was 3239. 1619 × 2 = 3238. The truncation really was writing exactly half of the same payload shape, which independently confirms the harness was doing what it claimed rather than something adjacent.

The gap

#	PREDICTION	OUTCOME
1	All three crash points leave the previous complete envelope intact; `load()` never returns partial and never throws	**Confirmed** across all three
2	`afterTempWriteBeforeReplace` is the real test	**Confirmed as the decisive case**, and it passed
3	Stale `.tmp` files accumulate and nothing cleans them	**Confirmed** – rounds 2 and 3 each left an orphan
4	Only the in-flight transition is lost	**Confirmed** – every round restored the state *before* the advance being written
5	Lowest confidence: whether `replaceItemAt` is genuinely atomic on watchOS	**Confirmed atomic.** No partial state was ever observed, at any injection point.

Five of five. The first time in this project that a full prediction set has held.

The harness was wrong at first, and the run that found it is kept

The initial rounds 1 and 2 were **invalid**. The crash-save button called `machine.start()` – beginning a fresh empty protocol – rather than `advance()`, so the payload being written was a ~264-byte empty state rather than a real transition.

How it surfaced: round 2's truncated temp file came back at **132 bytes** when roughly half of ~1869 was expected. The number was the only thing that looked wrong; both rounds otherwise reported clean, plausible results and would have been logged as passes.

What it cost: two things were wrongly scored before the fix.

- Prediction 4 was recorded as confirmed on the first round 1. It was not tested at all – the operation was a fresh start, not a transition. That scoring was mine and was wrong.
- The atomicity claim was being exercised against an unrepresentative payload.

After the one-line fix (`start()` → `advance()`), the same rounds produced 1619 and 3239 bytes – figures consistent with each other and with the real state size. The claim is now tested against what it was meant to be tested against.

Why this belongs in the record. The experiment nearly passed for the wrong reason. Nothing failed, no error appeared, and the only signal that anything was off was a byte count that did not fit. This is the same pattern the whole project has been documenting – a green result concealing a weaker test than intended – arriving this time inside the instrument built to detect it.

Cross-reference: this closes E2.2's prediction 3, previously recorded as *implemented but untested*.

E2.3 — Reconcile HealthKit's record with mine

Setup: after a recovery, compare the recovered session's `startDate` against the persisted semantic state's session start. Decide, in writing, which wins when they disagree.

My prediction

1. **They will not match exactly.** My state file is written a moment after the session starts, so I expect a difference of well under a second – but non-zero.
2. Under the source-of-truth policy: **HealthKit is authoritative for the session envelope** (start, end, heart rate, energy) and **I am authoritative for what happened inside it** (station identity, transition boundaries, notes).
3. Therefore station boundaries should be stored as **offsets from HealthKit's `startDate`**, not as independent absolute times. That way the two records cannot drift apart, and there is nothing to reconcile.
4. If prediction 3 holds, "reconciliation" mostly disappears as a problem – which would be a better outcome than solving it, and is worth testing before building a merge routine nobody needs.

Actual result – ROUND-TRIP: EXACT

Run 2026-08-24, 11:13. Workout session started, protocol started, four segments advanced, HealthKit session attached, then compared.

```
Semantic protocolStartedAt : 2026-08-24T04:12:17.032Z
HealthKit startDate        : 2026-08-24T04:12:05.668Z
HealthKit UUID             : unavailable-until-finished
Delta (semantic - HealthKit): +11364.458 ms
```

#	SEGMENT	STORED ABSOLUTE	OFFSET FROM HK START	RECOMPUTED
1	Preparing	04:12:17.032Z	+11.364458 s	04:12:17.032Z
2	Run 1	04:12:24.457Z	+18.789525 s	04:12:24.457Z
3	Roxzone 1	04:12:25.565Z	+19.897077 s	04:12:25.565Z
4	Station 1 – SkiErg	04:12:26.615Z	+20.946878 s	04:12:26.615Z

ROUND-TRIP: EXACT – every recomputed absolute time matched its stored value.

Offsets verified independently against the reported timestamps: 11.364, 18.789, 19.897, 20.947 s. All agree.

The gap

#	PREDICTION	OUTCOME
1	The two records will not match exactly; expect a difference well under a second	**Direction right, magnitude badly wrong.** The delta was **11.4 seconds**, not sub-second – off by more than an order of magnitude.
2	HealthKit authoritative for the envelope, the app for what happened inside it	**Upheld as a design position**, not empirically tested here. Recorded as a decision, not a result.
3	Boundaries stored as offsets from the HealthKit `startDate` round-trip without drift	**Confirmed** – **EXACT** across all four segments
4	If 3 holds, reconciliation mostly disappears rather than needing to be solved	**Confirmed.** With a shared origin there is nothing to merge.

Why prediction 1 was wrong is more interesting than the number. I assumed the gap would be precision error – sub-second clock or write latency. It is not. The 11.4 seconds is **operator delay**: the HealthKit session was started at 04:12:05 and the protocol at 04:12:17, two separate taps by a human.

So the delta is not noise to be tolerated, it is **semantic**. The two timestamps describe genuinely different events: when the workout began, and when the protocol began. Treating them as interchangeable – as a sub-second assumption invites – would silently misattribute eleven seconds of a race.

That sharpens prediction 2 from a preference into a requirement: HealthKit's `startDate` is the anchor because it marks the workout, and the semantic record's own start is a separate fact that must not be substituted for it.

Prediction 3 is the result that matters, and it holds. Offsets recomputed to the stored millisecond every time. With both records anchored to one origin, there is no drift to reconcile – the merge routine predicted to be avoidable is, in fact, avoidable.

Defect found during E2.3 — `start()` discards the HealthKit link

The first attempt returned *"no session is attached"* despite a session running. Cause: `ProtocolMachine.start()` constructs a fresh `WorkoutState` with no HealthKit fields, so starting a protocol **after** attaching silently wipes the anchor.

This is not merely an experiment-ordering inconvenience. In a real workout the sequence is naturally *start session → start protocol*, which is exactly the order that destroys the link – with no error, and no symptom until reconciliation is attempted much later.

The E2.1 design did not catch this. It specified the launch-recovery ordering rigorously, having been burned by it, but said nothing about the protocol-start path carrying the session reference. A rule learned in one place was not generalised to the neighbouring one.

Added to the design's open questions rather than patched, since the fix is a design decision: protocol start should either require a session or carry the existing one forward.

Order of work for L2: E2.0 first and alone. Its result decides whether E2.2's design survives contact with reality, and whether E2.3 is a real problem or an avoidable one.

CYCLE L2 · RESULTS SUMMARY

Status: COMPLETE. Days 3–4 (Aug 24–25), finished on Day 4.

Conditions for every measurement – Apple Watch Ultra 3 (watchOS 26.6), Xcode 26.6. Always-On Display off, app launched from the watch, **no debugger attached**, installed via CLI. Crash tests used deterministic injection inside the code rather than force-quit timing.

The question, and the answer

> What has to be true for a workout to survive the app dying?

Three things, all now measured:

1. **Accept that the app really dies.** watchOS does *not* resurrect a crashed app to sustain a workout session. There is no safety net.
2. **Recover exactly once, at launch, before anything creates a session.** `recoverActiveWorkoutSession` is not idempotent, and an "endpoint already exists" error is a self-inflicted ordering bug — never something to retry around.
3. **Persist timestamps atomically on every transition.** Proven to lose nothing across process death, and proven atomic at three separate crash points inside the write.

Results by experiment

EXPERIMENT	QUESTION	RESULT
E2.0	What happens when the app dies mid-workout?	**Central prediction FALSIFIED.** The system does not relaunch it. Two launches, both by hand.
E2.0b	Is `recoverActiveWorkoutSession` idempotent?	**No.** Call 1 recovered; call 2 failed against call 1's own session. **Solved E1.3.**
E2.1	The state machine, on paper	10 states, 26 transitions. Ordinary 6 states / 6 transitions; awkward 4 states / **20 transitions**.
E2.2	Persist on transition, then kill it	**Nothing lost.** Segments identical, elapsed carried through the dead interval.
E2.2b	Is the atomic write actually atomic?	**Yes — 5 of 5 predictions confirmed.** Survived all three injected crash points.
E2.3	Reconcile HealthKit's record with mine	**ROUND-TRIP: EXACT.** Offsets remove reconciliation entirely.

The two headline measurements

Persistence survives death. Killed during Run 2 with four segments complete; on relaunch every segment duration was identical and the current state's elapsed clock had continued through the interval the process did not exist for — 21.4 s before the kill, 36.0 s at restore.

The write is genuinely atomic. Crash before the temp write, during it (leaving a truncated 1619-byte file), and after it but before the replace (leaving a **complete 3239-byte** file beside a 2552-byte committed one). In every case the committed envelope loaded intact and the newer temp file was correctly ignored.

Prediction tally

EXPERIMENT	CONFIRMED	WRONG	UNTESTED / OTHER
E2.0 (5)	1	**2**	1 untested, 1 undecided
E2.0b (5)	3	0	1 refined, 1 unneeded
E2.1 (4)	3	0	1 mixed
E2.2 (5)	1	**1**	3 untested (2 later closed by E2.2b)
E2.2b (5)	**5**	0	0
E2.3 (4)	2	**1**	1 design position

Four wrong predictions, and three of them were the most valuable results in the cycle:

- **E2.0 predictions 1 and 2** — the background-relaunch hypothesis. Killing it forced the search that found the real mechanism.
- **E2.2 prediction 5** — expected a boring first-run failure; there wasn't one.
- **E2.3 prediction 1** — expected a sub-second delta; got 11.4 seconds, and the reason (operator delay, not precision error) changed the source-of-truth rule from a preference into a requirement.

Architectural rules L2 established

1. **The app is not resurrected.** Design for real termination; there is no system safety net for an active workout session.
2. **Recovery is a one-shot launch decision.** Call once, before any session can be instantiated; retain the result; never call again. "Endpoint already exists" means a session was created before recovery ran.
3. **Persist on every transition, atomically** — temp file then replace. Verified against three crash points; a complete-but-uncommitted temp file is correctly ignored.
4. **Store timestamps, never durations.** L1's rule extended from a *suspended* process to a *dead* one.
5. **Anchor boundaries as offsets from HealthKit's `startDate`.** Round-trips exactly, and removes reconciliation as a problem rather than requiring it to be solved.

6. **HealthKit's `startDate` and the protocol's start are different events.** The gap is semantic — operator delay — not noise, and the two must never be substituted for one another.

Defects found, and where they came from

DEFECT	FOUND BY	NATURE	
`start()` discards the HealthKit link	E2.3's first run	**Design gap.** The natural order — session then protocol — is the one that breaks it, silently.	
Crash-save button called `start()` not `advance()`	An anomalous **132-byte** temp file	**Harness defect.** Both prior rounds had reported clean, plausible passes.	
A "missing rationale" reported in the E2.1 design	Re-reading the design	**False finding of mine** — a grep using `\`	`alternation under` -E` returned 0 and the zero was trusted.
E2.1 specified launch-recovery ordering but not protocol-start ordering	E2.3	A rule learned in one place was not generalised to the neighbouring one.	

The pattern, now inside the instruments

L1 recorded six cases of a success signal concealing a failure. L2 added four more, and they moved inward:

- `App installed:` reported success on an install that had silently not replaced the binary
- The E2.2b harness passed twice while testing a 264-byte empty state instead of a real transition
- A broken grep returned 0 and was read as evidence of absence
- The E2.1 design was audited as sound while containing an unnoticed ordering gap

In L1 the tooling misled. In L2 the instruments built to catch that did. The transferable rule is unchanged and now better evidenced: **a tool's output is evidence about the tool as much as about the subject**, and a number that does not fit is worth more attention than a result that does.

Open questions carried into L3

1. Should starting a protocol **require** an active session, or carry an already-attached one forward?
2. Stale `.tmp` files accumulate with no cleanup policy. Harmless for correctness; unbounded over time.
3. Does `activityType` affect session longevity? *(open since L1)*
4. What is the battery cost of a 90-minute session? *(open since L1)*

A scoping note for L3. L3 asks which half of the record HealthKit owns and which is mine. **E2.3 has already answered much of it** — HealthKit owns the envelope, the app owns what happened inside it, and offsets bind them without drift. L3 may be better spent on what that ownership split makes *possible* — whether the workout genuinely crosses to the phone unaided, and what arrives when it does — than on re-deciding a boundary already settled by measurement.

CYCLE L3 · The crossing — Days 5–6 (Aug 27–28) · MIDPOINT

Learning question (RESCOPED 2026-08-24): *Does a workout actually cross to the phone unaided, and what arrives when it does?*

Why rescoped. L3 originally asked which half of the record HealthKit owns and which is mine. **E2.3 largely answered that by measurement**: HealthKit owns the envelope, the app owns what happened inside it, and offsets bind them without drift. Re-deciding a settled boundary would be busywork.

What remains genuinely untested is the assumption this entire project was reframed around on Day 1 — that **HealthKit carries the workout to the phone on its own**. Everything in L4 depends on it, and nothing has ever verified it. If it is false, L4 is a much larger problem than planned. If it is true, the question becomes what *else* can ride along.

Predictions below were written 2026-08-24, before any L3 code existed.

E3.0 — Does the workout cross at all, unaided?

Setup: finish a workout on the watch. Do not write any transport code. Watch for it on the iPhone — first in Apple's Health app, then in our own iOS app querying `HKWorkout`.

My prediction

1. **It crosses with no code of mine.** This is the Day 1 reframe and it is still an assumption. I expect it to hold, because the whole architecture was rebuilt around it.
2. **Not instantly.** I expect seconds to minutes, not sub-second. Sync is opportunistic, not a push.
3. **Conditions will matter** — phone nearby, possibly unlocked, possibly on the same network. If it only crosses under conditions a HYROX athlete would not meet mid-race, "it works" is misleading.
4. Once present, it is queryable by our own iOS app, not merely visible in Apple's Health app — same store, same data.
5. **Highest-stakes prediction in the project so far.** Every architectural decision since Day 1 rests on it, and it has never been tested.

Actual result — IT CROSSES. Even locked. Even from another room.

Run 2026-08-26, 09:12–09:28. Three workouts finished on the watch from HyroxCoach. No transport code of any kind exists in this project.

#	START	END	DURATION	CONDITION	CROSSED?
A	09:12:22	09:13:24	1m 1s	Phone unlocked, nearby	Yes
B	09:14:24	09:15:25	1m 0s	**Phone locked**	Yes
C	09:16:16	09:17:27	1m 10s	**Phone in another room**	Yes

All three appear in the iOS app as `Cross Training — HyroxCoach — Apple Watch`, with UUIDs, start, end, duration and active energy intact.

Measured latency: ≤ 14 seconds, and it was measured on the *locked* case. Workout B ended at 09:15:25; the list refreshed at 09:15:39 already showed it, reporting "14s since workout ended". The true crossing time is somewhere at or under that.

The gap

#	PREDICTION	OUTCOME
1	It crosses with no code of mine	**Confirmed.** The Day 1 reframe holds.
2	Not instant — seconds to minutes	**Confirmed** — ≤14 s, at the fast end of the range
3	Conditions will matter: phone nearby, possibly unlocked	**FALSIFIED.** It crossed with the phone locked, and again with the phone in another room.
4	Queryable by my own app, not just visible in Apple's Health app	**Confirmed** — read directly from `HKWorkout` by HyroxCoach on iOS
5	Highest-stakes prediction in the project	**Held.** Every architectural decision since Day 1 rested on this, and it is now measured rather than assumed.

Prediction 3 being wrong is the most consequential result of the cycle. I expected the crossing to be conditional — phone awake, phone close — and warned that "it works" would be misleading for an athlete whose phone sits in a locker for ninety minutes. It is not conditional. A locked phone in another room received the workout within seconds.

That removes a risk that had been sitting under the whole architecture unexamined. The concern was legitimate; the answer is simply better than expected.

Note on what this does and does not measure. "14s since workout ended" is *time since the workout ended*, not a measured sync delay — they are only equal if the query happens the instant it arrives. So 14 s is an **upper bound**, not a measurement of the crossing itself. Establishing the true latency would need an observer query timestamping arrival, which is not built.

E3.1 — What actually arrives?

Setup: compare what the watch recorded against what the phone can see. Enumerate the arriving `HKWorkout` field by field.

My prediction

1. **The physiological envelope survives**: `startDate`, `endDate`, `duration`, heart-rate samples, active energy.
2. `activityType` survives as the approximation chosen — `.crossTraining`. HYROX has no dedicated type, so the record will describe the workout slightly wrongly and permanently.
3. **None of my semantic structure survives.** No stations, no roxzone boundaries, no segment identities. HealthKit has nowhere to put them.
4. Therefore the phone will show a workout of the right length with no idea what happened inside it — which is exactly the gap L4 exists to close.

Actual result — the envelope arrives, the meaning does not

From the same three workouts, 2026-08-26.

FIELD	ARRIVED?	VALUE SEEN
`UUID`	Yes	e.g. `319DF277-DD0A-4A2E-8C19-E24AC7212DD1`
`startDate` / `endDate`	Yes	09:12:22 → 09:13:24
`duration`	Yes	1 minute, 1 second
`activityType`	Yes	**Cross Training**
Total active energy	Yes	3.8 / 3.78 / 4.39 kcal
Source	Yes	**HyroxCoach**
Device	Yes	**Apple Watch**
Metadata	Only Apple's own	**`HKIndoorWorkout = 1`** – nothing else
Station identity, roxzone boundaries, segment structure	**No**	absent entirely

The gap

#	PREDICTION	OUTCOME
1	The physiological envelope survives	**Mostly confirmed** – start, end, duration and active energy all arrived. **Heart-rate samples not verified**: the detail view reports energy but does not enumerate HR samples, so that part is untested.
2	`activityType` survives as the chosen approximation, `.crossTraining`	**Confirmed.** The phone reports "Cross Training" – a permanent, slightly wrong description of a HYROX session, since HealthKit has no HYROX type.
3	None of my semantic structure survives	**Confirmed.** The only metadata present is `HKIndoorWorkout`, set by Apple. Not one field of mine crossed.
4	The phone shows a workout of the right length with no idea what happened inside it	**Confirmed exactly.**

This is the gap L4 exists to close, now shown rather than assumed. The phone knows a cross-training workout happened, when, for how long, and how much energy it cost. It does not know a SkiErg was involved, or that there was a roxzone, or that any of it was HYROX. The envelope crosses free; the meaning does not.

Untested: whether heart-rate samples crossed. The instrument reports energy but not HR sample counts, so this remains open.

E3.2 — Can custom metadata ride along?

The experiment that could reshape L4.

Setup: attach a metadata dictionary to the workout at `finishWorkout` time, containing a compact encoding of the semantic record – station names and their offsets from `startDate`. Check whether it survives the crossing and is readable on the phone.

My prediction

1. `HKWorkout` accepts a metadata dictionary of property-list types, and **it crosses with the workout**. If so, some semantic data can travel on HealthKit's own sync rather than needing WatchConnectivity at all.
2. **But it can only be set once, at finish.** It cannot be updated per transition, so it carries a *summary*, never a live stream.
3. There will be practical limits – size, and permitted value types – that are not clearly documented and must be found by testing.
4. **If 1 and 2 hold, L4 shrinks substantially**: the phone would already receive a complete post-workout record, and WatchConnectivity would only be needed for live mid-workout data, which the iOS side does not display anyway.
5. **Lowest confidence in this cycle.** I do not know the size limits, whether nested structures survive, or whether metadata is treated as first-class during sync. This is the one most likely to fail in an interesting way.

Actual result – THE METADATA CROSSES.

Run 2026-08-26, 10:38–10:39. Workout finished on the watch with the metadata toggle on. No transport code exists in this project.

Workout `F0DE085D-9774-409A-80C1-6F4B41FF597E`, 42 s, Cross Training, source HyroxCoach, device Apple Watch.

Metadata as read on the **iPhone**:

```
HKIndoorWorkout           1
HYROXProtocolStartOffset  17.355
HYROXSchemaVersion        1
HYROXSegmentCount         4
HYROXSegments             Preparing@17.355;Run 1@19.090;
                          Roxzone 1@20.073;Station 1 — SkiErg@21.422
```

All four custom keys arrived intact and complete. Nothing was truncated, renamed, or dropped. `HYROXProtocolStartOffset` matches the `Preparing` offset exactly, as it should, since the protocol begins with that state.

The gap

#	PREDICTION	OUTCOME
1	`HKWorkout` metadata crosses with the workout	**Confirmed.** Written on the watch, read on the phone, with no code of mine moving it.
2	It can only be set once, at finish — a summary, never a live stream	**True by construction**, since it is attached at `finishWorkout`. Not adversarially tested.
3	There will be practical limits on size and value types	**Not reached at this scale.** Four segments produced a short string. **Untested at full race length.**
4	If 1 and 2 hold, L4 shrinks substantially	**Confirmed.** The phone now receives a complete post-workout semantic record with no transport code.
5	Lowest confidence in the cycle; most likely to fail in an interesting way	**Wrong — it did not fail at all.** I expected this to break and it worked first time.

What this changes. E3.1 showed the phone receiving a workout of the right length with no idea what happened inside it. E3.2 closes that gap using the same free sync: station names and their offsets now travel with the workout. Combined with E2.3's proof that offsets round-trip exactly, the phone can reconstruct every boundary as an absolute time without a merge step.

The size limit is the one real caveat. This test encoded **4 segments**. A full HYROX race is 8 runs, 8 roxzones and 8 stations — around **25 segments**, producing a string roughly six times longer. HealthKit's metadata limits are not clearly documented, and a limit could appear as silent truncation rather than an error. **Until that is tested at full length, "metadata crosses" is proven only for short workouts.**

Consequence for L4. L4 was planned as transport work — WatchConnectivity, delivery, duplicates, deduplication. Most of that now has no purpose: the post-workout record already arrives. What remains is live mid-workout data, which the iOS side does not display. **L4 should be rescoped before it starts, in the same way L3 was.**

Order of work: E3.0 first and alone — if the workout does not cross, E3.1 and E3.2 are meaningless and the cycle becomes about why not.

CYCLE L3 · RESULTS SUMMARY

Status: COMPLETE. Run 2026-08-26, ahead of its Aug 27–28 slot.

Conditions — Apple Watch Ultra 3 (watchOS 26.6), iPhone 16 Pro Max (iOS 26.6.1). Workouts finished from HyroxCoach on the watch, read by HyroxCoach on the phone. **No transport code exists anywhere in this project.**

The question, and the answer

> `Does a workout actually cross to the phone unaided, and what arrives when it does?`

Yes, and more arrives than expected. A workout finished on the watch reaches the phone within seconds, with the phone **locked** and **in another room**. The physiological record crosses on its own, and custom metadata crosses with it — so the app's own semantic record can travel the same way.

Results by experiment

EXPERIMENT	QUESTION	RESULT
E3.0	Does it cross at all, unaided?	**Yes.** Three workouts, three conditions, all crossed. **≤14 s**, measured on the locked case.
E3.1	What arrives?	The envelope: UUID, start, end, duration, activity type, energy, source, device. **No semantic structure.** Only metadata was Apple's `HKIndoorWorkout`.
E3.2	Can custom metadata ride along?	**Yes.** All four HYROX keys arrived intact, including the full segment string.

The crossing, measured

CONDITION	CROSSED?
Phone unlocked, nearby	Yes
Phone locked	Yes
Phone in another room	Yes

Prediction 3 of E3.0 was falsified, favourably. I expected the crossing to depend on the phone being awake and close, and warned that would make it useless for an athlete whose phone sits in a locker. It does not depend on either.

Prediction tally

EXPERIMENT	CONFIRMED	WRONG	OTHER
E3.0 (5)	4	**1** (falsified favourably)	–
E3.1 (4)	3	0	1 partial (HR samples unverified)
E3.2 (5)	3	**1** (expected failure, got success)	1 true by construction

Both wrong predictions were pessimistic and both were wrong in the useful direction – the crossing is less conditional than feared, and metadata worked first time where I expected it to break.

What L3 established

1. **The Day 1 reframe is true.** HealthKit carries the workout from watch to phone with no code of mine. Every architectural decision since Day 1 rested on this, and it is now measured.
2. **It is not conditional** on the phone being unlocked or nearby.
3. **The envelope crosses free; the meaning does not** – HealthKit has nowhere to put station identity or roxzone boundaries.
4. **But metadata does cross**, so the semantic record can ride the same sync. Combined with E2.3's exact offset round-trip, the phone can rebuild every boundary with no merge step.
5. `activityType` is a permanent approximation. HYROX has no HealthKit type, so the record will always describe the workout slightly wrongly.

Untested, and it matters

The metadata size limit. E3.2 encoded **4 segments**. A full HYROX race is roughly **25** – 8 runs, 8 roxzones, 8 stations, plus preparation – producing a string about six times longer.

HealthKit's metadata limits are not clearly documented, and a limit would most likely appear as **silent truncation rather than an error**. Until tested at full length, the result is proven **only for short workouts**.

Heart-rate sample arrival also remains unverified; the iOS instrument reports energy but does not enumerate HR samples.

Consequence: L4 must be rescoped

L4 was planned as transport work – WatchConnectivity, delivery, duplicate handling, deduplication. **Most of that now has no purpose.** The complete post-workout record already arrives by itself. What remains is live mid-workout data, which the iOS side does not display.

This is the second time a measurement has dissolved planned work rather than completing it: E2.3 removed reconciliation, E3.2 has removed most of transport.

CYCLE L4 · The limits of the free crossing — Days 7–8 (Aug 31 – Sep 1)

Learning question (RESCOPED 2026-08-26): *Where does the free crossing break, and what genuinely still needs a transport of my own?*

Why rescoped. L4 was planned as transport work: `WCSession` transfer modes, delivery guarantees, duplicate handling, deduplication by stable ID. **E3.2 removed the reason for most of it.** The complete post-workout record – envelope plus semantic structure – already reaches the phone on HealthKit's own sync, with no code of mine.

Building a transport for data that already arrives would be work done to match a plan rather than to meet a need. What is genuinely unknown is **where that free crossing stops working**, and what, if anything, is left over.

Predictions written 2026-08-26, before any L4 code existed.

E4.0 — Does the metadata survive a full-length race?

Setup: encode a realistic full HYROX protocol — 8 runs, 8 roxzones, 8 stations, plus preparation, around **25 segments** — into the same metadata keys, finish a workout, and read it on the phone. Compare the string received against the string sent, character for character.

My prediction

1. **It will still cross, but this is where a limit would appear.** E3.2's string was 4 segments; this is roughly six times longer.
2. **If a limit exists, it will truncate silently rather than error.** That is the failure mode this project has met repeatedly, and the one that would be most damaging here — a partial station list looks like a complete one.
3. **Therefore the test must compare, not merely observe.** "The metadata is present" is not evidence; only a character-for-character match against what was sent is.
4. If it survives at 25 segments, no realistic HYROX workout will exceed it, and the free crossing carries the whole product.
5. **Least confident:** whether the limit is on a single value, the whole dictionary, or the number of keys. Each would need a different workaround.

Actual result — INTACT at full length. And the verdict is wrong about something else.

Run 2026-08-27, 10:24–10:25. Full race seeded, workout finished with metadata attached, read on the iPhone.

```
HYROX Integrity: INTACT
Received segment-string character count: 601

HYROXSchemaVersion        1
HYROXSegmentCount         25
HYROXSegmentsLength       601
HYROXProtocolStartOffset  -85543.894
HYROXSegments             Preparing@-85543.894;Run 1@-85483.894;
                          Roxzone 1@-85108.894;Station 1 — SkiErg@-85068.894;
                          … Station 8 — Wall Balls@-80258.894
```

On size, the prediction holds. All 25 segments crossed. All eight stations arrived in the correct order — SkiErg, Sled Push, Sled Pull, Burpee Broad Jumps, Rowing, Farmers Carry, Sandbag Lunges, Wall Balls. 601 characters sent, 601 received. Nothing truncated.

But every offset is negative, and the check said INTACT anyway

Every segment reports a time **before the workout started** — `Preparing@-85543.894` is 23.76 hours earlier. That is impossible for real data.

Cause, confirmed by arithmetic. `seedFullRace()` did not reset `protocolStartedAt`. It reused the value persisted from the previous session on 26 August at ~10:38, while the new workout session began on 27 August at 10:24:24. The gap between those two instants is **85,543 seconds** — matching the reported offset exactly. The seeded segments were anchored to a stale protocol start.

The integrity check reported INTACT on impossible data. It compares the received length and segment count against what was declared, and both matched, so it passed. It never asks whether the values *make sense*.

This is the same failure this project keeps recording, and this time it is in the instrument I built specifically to catch it. E4.0's own prediction 3 said "the metadata is present" is not evidence, and that only a character-for-character comparison counts. That was right as far as it went — and insufficient. A complete payload of nonsense passes a completeness check.

Completeness and correctness are different properties. INTACT means nothing was lost in transit. It does not mean the data was right when it left.

The gap

#	PREDICTION	OUTCOME
1	It will still cross, but this is where a limit would appear	**Confirmed on crossing, limit not reached.** 601 characters, 25 segments, all intact.
2	If a limit exists it will truncate silently	**Not reached.** No truncation at 601 characters, so the failure mode is still untested.
3	The test must compare, not merely observe	**Right, and not enough.** The comparison worked and correctly reported the length matched. It still passed data that was obviously wrong.
4	If it survives at 25 segments, the free crossing carries the whole product	**Confirmed for size.** No realistic HYROX workout exceeds 25 segments.
5	Least confident: whether the limit is on one value, the whole dictionary, or the key count	**Still unknown.** The ceiling was never found because 601 characters did not approach it.

What is now established: a full-length HYROX payload crosses to the phone complete, with no transport code. The size question is answered for any realistic race.

What is not: where the actual limit sits. 601 characters passed comfortably; the ceiling could be far higher. E4.1 was written to run only if E4.0 failed — it did not fail, so finding the true limit is now optional rather than necessary.

Two defects found

DEFECT	EFFECT
`seedFullRace()` does not reset `protocolStartedAt`	Seeded segments anchor to a stale protocol start from a previous session, producing negative offsets
The integrity check tests completeness only	A payload can be complete and still be nonsense. INTACT was reported for segments dated before the workout began

The second is the more important one. A plausibility check — offsets must be non-negative and must increase — would have caught this immediately, and would catch a real ordering or anchoring bug in production.

E4.1 — If it breaks, where exactly?

Run only if E4.0 fails.

Setup: increase the payload until it breaks, then narrow down the boundary. Record whether failure is an error, a silent truncation, or a refusal to save the workout at all.

My prediction

1. Failure will be silent — the workout saves, the metadata is short.
2. The limit will be on the **total dictionary size**, not the key count.
3. A workaround exists: split the segment string across several numbered keys, since the total is what matters, not any single value.
4. If instead the whole workout fails to save, that is far more serious — an annotation would be destroying the record it describes, and metadata would have to become optional and verified.

Actual result — NOT RUN, by design.

E4.1 was written with an explicit condition: *"Run only if E4.0 fails."* E4.0 did not fail. A full-length payload of 25 segments and 601 characters crossed complete, and no truncation limit was reached.

The gap

Its four predictions are therefore untested, and remain so deliberately rather than by omission:

#	PREDICTION	STATUS
1	Failure will be silent — the workout saves, the metadata is short	**Untested** — no failure occurred
2	The limit will be on total dictionary size, not key count	**Untested**
3	A workaround exists: split the segment string across numbered keys	**Untested**
4	If the whole workout fails to save instead, that is far more serious	**Untested**

Why this is recorded rather than deleted. The experiment was designed as a contingency and the contingency did not arise. Removing it would hide the fact that the failure mode was anticipated and prepared for, which is part of the evidence of how the cycle was planned.

What remains unknown as a result: where HealthKit's metadata ceiling actually sits. 601 characters passed comfortably; the limit was never approached. This is carried in the L4 decision record as an accepted risk.

E4.2 — What is genuinely left for a transport of my own?

Setup: list what the iOS app needs that metadata cannot carry, and check each against what is already arriving.

My prediction

1. **Almost nothing is left.** The post-workout record is complete once E4.0 passes.
2. The only candidate is **live data during the workout** — current station, elapsed time, heart rate on the phone while training. **The iOS app does not display any of it**, and the product's own division of responsibility says the phone is for after the workout.
3. Therefore the honest answer is likely that **WatchConnectivity is not needed for this product at all**, and the right output of L4 is a written justification for not building it.
4. **A decision not to build something is a legitimate result**, provided it is evidenced. Building `WCSession` transport to match the original plan would be the worse outcome.
5. **Risk to watch for:** wanting to build the transport because it was planned, or because it is the more impressive thing to demonstrate. That is not a technical reason.

Actual result — nothing is left. Decision recorded: do not build the transport.

2026-08-27. Written up as an architecture decision record at `design/L4-transport-decision.md` (95 lines, status Accepted).

Decision: do not build a watch-to-phone transport for this product; use HealthKit's own sync to carry the completed workout and its semantic metadata.

Capability the transport was planned to provide, against what measurement showed:

PLANNED CAPABILITY	STILL NEEDED?	SETTLED BY
Move the completed workout to the phone	No — it already arrives	E3.0
Move station identity and boundaries	No — metadata carries them	E3.2, E4.0
Handle a full-length race payload	No — 25 segments, 601 characters, intact	E4.0
Reconcile the two records	No — offsets share one anchor and round-trip exactly	E2.3
Deliver while devices are apart	No — crossed with the phone locked and in another room	E3.0
Deliver live data mid-workout	**Not required by the product** — the iOS app does not display it	product scope

The gap

#	PREDICTION	OUTCOME
1	Almost nothing is left once E4.0 passes	**Confirmed**
2	The only candidate is live mid-workout data, which iOS does not display	**Confirmed.** Metadata is attached once at `finishWorkout`, so it cannot carry live data — but nothing in the product needs it to.
3	The honest answer is likely that WatchConnectivity is not needed at all, and the right output is a written justification for not building it	**Confirmed.** That justification is the ADR.
4	A decision not to build is a legitimate result if evidenced	**Upheld.** Every claim in the ADR traces to a measurement in this workbook.
5	Risk: wanting to build it because it was planned, or because it demonstrates better	**Recorded explicitly in the ADR**, as its own section, so a future reader can see the temptation was considered rather than avoided by accident.

Reversal conditions are written down and checkable, so this is a decision rather than a permanent conclusion. Among them: the iOS app gaining a live in-workout screen; any requirement that data reach the phone before the workout ends; a real payload arriving truncated; or a user with iCloud Health sync disabled. The ADR states plainly that the existence of WatchConnectivity is not itself a reason to use it.

Three risks accepted, all recorded:

1. HealthKit's metadata size limit was never reached, so it remains unknown. 601 characters passed comfortably; the ceiling is untested.
2. The crossing depends on the user having iCloud Health sync enabled between watch and phone. **This was never tested with it off.**
3. The 14-second figure is an upper bound on *time since the workout ended*, not a measured sync delay.

What this means for the cycle. L4 was planned as the largest build in the project. It produced no transport code, and that is the correct outcome. The work it was meant to do had already been removed by measurement — first by E2.3, then by E3.2 and E4.0.

Order of work: E4.0 first. It decides whether the free crossing carries the whole product or only part of it, and therefore whether E4.1 is needed at all.

CYCLE L4 · RESULTS SUMMARY

Status: COMPLETE. Run 2026-08-27, ahead of its Aug 31 – Sep 1 slot.

Conditions — Apple Watch Ultra 3 (watchOS 26.6), iPhone 16 Pro Max (iOS 26.6.1). Payload seeded as test data, attached at `finishWorkout`, read on the phone.

The question, and the answer

> `Where does the free crossing break, and what genuinely still needs a transport of my own?`

It did not break, and nothing is left to build. A full-length race payload crosses complete. Every capability the planned transport was meant to provide has been removed by measurement.

Results by experiment

EXPERIMENT	QUESTION	RESULT
E4.0	Does metadata survive a full-length race?	**Yes.** 25 segments, 601 characters sent and received, all eight stations in order. No truncation.
E4.1	If it breaks, where exactly?	**Not run.** It was written to run only on failure. E4.0 did not fail.
E4.2	What still needs a transport of my own?	**Nothing.** Decision recorded: do not build it.

The decision

Do not build a watch-to-phone transport. Use HealthKit's own sync to carry the completed workout and its semantic metadata. Recorded at `design/L4-transport-decision.md`, status Accepted, with reversal conditions and accepted risks.

L4 was planned as the largest build in the project and produced **no transport code**. That is the correct outcome: the work had already been made unnecessary, first by E2.3, then by E3.2 and E4.0.

Prediction tally

EXPERIMENT	CONFIRMED	NOT REACHED	NOTES
E4.0 (5)	3	2	The size limit was never approached, so the truncation failure mode is still untested
E4.2 (5)	5	0	Including the prediction that the right output was a justification, not a build

---

The defect E4.0 exposed, which matters more than the size result

Every offset in the payload was **negative** — each segment dated about 23.76 hours before the workout began. The cause was arithmetic-exact: `seedFullRace()` reused `protocolStartedAt` from the previous day's session while the workout had just started, and the 85,543-second gap between them matched the reported offset precisely.

The integrity check reported INTACT anyway. It compared declared length and segment count, both matched, so it passed. It never asked whether the values were possible.

Completeness and correctness are different properties. INTACT meant nothing was lost in transit. It said nothing about whether the data was right when it left. E4.0's own prediction had said "the metadata is present" is not evidence and only a character-for-character comparison counts — that was right, and insufficient. A complete payload of nonsense passes a completeness check.

This is the ninth instance of the project's recurring pattern, and the closest to home: it was in the instrument built specifically to catch that pattern.

Both defects fixed

DEFECT	FIX
`seedFullRace()` anchored to a stale protocol start	Now takes the running session's start and lays segments forward from it. Seeding with no session running is refused with an explanation rather than substituting `Date()`.
The integrity check tested completeness only	Now tests plausibility after completeness: offsets must be non-negative, strictly increasing, within the workout's duration, and parseable. A complete-but-impossible payload reports **IMPLAUSIBLE** and names the offending value.

Re-run 2026-08-27, 11:00 — both fixes verified on device

Workout `08326EBA-6392-43DF-A226-6AE467F4A0C0`, 27 Aug 11:00:00 → 11:01:12, duration **72.4 s**.

```
HYROX Integrity: IMPLAUSIBLE: final offset 5285.000 exceeds workout
                 duration 72.43331408500671 by more than 60 seconds
First and last offsets: first 0.000; last 5285.000
Received segment-string character count: 543
HYROXSegmentCount 25   HYROXSegmentsLength 543
HYROXProtocolStartOffset 0.000
```

Fix 1 — the seed anchor — works. Offsets now run from `0.000` to `5285.000`. Verified independently: 25 segments, all non-negative, strictly increasing, spanning 88.1 minutes. The previous run had every offset at roughly −85,543. That defect is gone.

Fix 2 — the plausibility check — works, and fired on a rule I had not expected it to. It did not reject the offsets for being negative or out of order, because they no longer are. It rejected them because the last segment sits **5,285 seconds** into a workout that lasted **72.4 seconds**. Segments cannot extend 88 minutes past a 72-second workout.

That verdict is correct, and it is a property of the test rather than of the product. `seedFullRace` writes a realistic 90-minute race, but the workout it is attached to lasts about a minute. The check is right to refuse it. In a real race the workout would last roughly as long as the segments describe and the rule would pass.

The size result still stands, and is now cleaner. Completeness passed: 543 characters sent, 543 received, 25 segments, all eight stations in order. The earlier run measured 601 characters only because the negative offsets were longer strings. Both runs crossed complete.

What this re-run actually demonstrates. A verdict of `INTACT` would have proved very little — a check that always says INTACT would produce it too. Instead the check **refused a payload**, named the failing rule, and quoted both numbers. That is evidence it can say no, which is the only thing that makes it saying yes worth anything.

Still to confirm: the two supporting checks from the same re-run — that the pre-fix workout from 10:24 now reports IMPLAUSIBLE rather than INTACT, and that seeding without a running session is refused. Reported as run; not yet recorded here with their exact output.

Risks accepted, carried forward

1. **HealthKit's metadata size limit is unknown.** 601 characters passed comfortably; the ceiling was never found.
2. **The crossing was never tested with iCloud Health sync disabled.** The entire architecture depends on that setting being on.
3. **The 14-second figure is an upper bound**, not a measured sync delay.

CYCLE L5 · Honest representation — Days 9–10 (Sep 3–4)

Learning question: What can this data honestly say, and what can it not?

Opened 2026-08-27, ahead of schedule. Predictions written before any L5 code existed.

What L5 has to close. Four cycles have proved the pipeline works. None of them produced a real workout. Every session so far has been seeded test data or about one minute of sitting still, and the review screen on the phone is a diagnostic instrument, not something a person would read after training. Three things are still outstanding, and this is the last cycle to do them.

E5.0 — A real training session

Nothing in this project has yet recorded actual exercise.

Setup: perform the shortened protocol — 2 runs and 2 stations — as **real physical effort**, advancing the state by hand at each boundary, with the metadata toggle on. Then read it on the phone.

My prediction

1. **The plausibility check will report INTACT for the first time on real data.** Every prior run failed it, or passed it wrongly. Here the segment offsets will genuinely fit inside the workout's duration, because the workout and the segments describe the same event.
2. **Heart rate will be meaningfully elevated.** Every reading so far has been about 84 bpm from sitting at a desk. Real effort should produce something clearly different, which also confirms the live builder is collecting during movement rather than only when idle.
3. **Tapping to advance while out of breath will be harder than the desk tests suggest.** I expect at least one mis-tap or hesitation, which is exactly what the undo history in the state machine design was written for and has never been exercised in anger.
4. **Something will go wrong that no seeded test predicted.** Four cycles of desk testing have never involved sweat, movement, or a wrist in motion.
5. **Lowest confidence:** whether the wrist-down and screen-off behaviour measured in L1 holds while actually moving, as opposed to lying still on a table.

Actual result — REAL SESSION COMPLETED.

Run 2026-08-31, 11:17:14 → 11:21:01. Workout `C1F8E587-6013-4AA0-92DF-785AC67A53BD`*, duration **3 m 47 s**. Stations substituted: SkiErg → burpees, Sled Push → walking lunges.*

```
HYROX Integrity: INTACT
First and last offsets: 5.262 → 172.688
Segment string: 134 characters, 7 segments
Total active energy: 33.43 kcal
```

SEGMENT	DURATION
Preparing	17.47 s
Run 1	**58.42 s**
Roxzone 1	24.76 s
Station 1 — burpees	**32.44 s**
Run 2	**32.30 s**
Roxzone 2	**2.03 s** ← unexplained
Station 2 — walking lunges	**54.31 s**

Effort is unambiguous. 33.43 kcal across 227 s is **8.8 kcal/min**, against **3.3 kcal/min** for the desk run that preceded it — roughly 2.7× the intensity, and 17× the total energy. These are exercise durations: runs measured in tens of seconds, stations in half-minutes.

The gap

#	PREDICTION	OUTCOME
1	The plausibility check reports INTACT for the first time on real data	**Confirmed.** Offsets non-negative, in order, and inside the workout's duration — now on a genuine 3m47s session.
2	Heart rate will be meaningfully elevated	**Confirmed directly, 2026-09-01.** A second real session recorded **51 heart-rate samples, 72–140 bpm, average 111.9**. Every desk test read about 84 bpm. The maximum is 67% above resting. The instrument was added specifically to close this by measurement rather than by inference from energy.
3	Tapping while out of breath will be harder than desk tests suggest	**Confirmed.** Roxzone 2 recorded **2.03 s** against 24.76 s for Roxzone 1 in the same session. Cause confirmed by the learner: **Advance was tapped twice by accident.** The second tap ended the roxzone almost as soon as it began.
4	Something will go wrong that no seeded test predicted	**Confirmed.** Five cycles of desk testing never produced a mis-tap. The first session involving real effort produced one within four minutes.
5	Least confident: whether L1's wrist-down behaviour holds while moving	**Confirmed, 2026-09-01.** During the second real session the arm was kept down for the whole of Run 2 — **89.8 s**, the longest segment recorded — with no interaction. The segment recorded correctly and the session continued. L1 measured this on a desk with the watch lying still; it holds while running.

The 2.03-second roxzone is the most interesting number in the session. Every other segment is consistent with what was physically done. This one is not, and the integrity check passed it — because 2.03 s is non-negative, in order, and inside the workout. **It is plausible by the rules and wrong in fact.**

That is the same lesson as E4.0's INTACT verdict, arriving now from real exercise rather than seeded data: a check can only test the properties it was given, and "is this number possible" is not the same question as "is this number true". A mis-tap produces a perfectly well-formed lie.

This directly feeds E5.1. A station or roxzone duration is not a recorded fact. It is the difference between two timestamps written when a human, mid-effort, managed to press a button. Every such number inherits that.

The undo history was designed for exactly this, and was not used

L2's state machine design identified accidental advance as the hardest path to model, and specified a bounded undo history holding complete state snapshots so a mis-tap could be reversed **with its original timestamps** rather than replaced by a fresh one. That mechanism was built and persisted. E5.0 was its first real opportunity.

It was not reached for. The mis-tap happened, the run continued, and the wrong duration was recorded and shipped to the phone.

That is not a fault in the undo logic, which works. It is a finding about where the control lives: undo sits in the E2.2 diagnostic section, not in the workout flow, and nothing on the screen at the moment of the mistake offered to fix it. A correction mechanism that is not reachable at the moment of the error is not available in practice.

The consequence for the product: correction has to be part of the live workout interface, not a separate screen. On a watch, mid-effort, out of breath, it must be one obvious action or it will not be used — as just demonstrated.

Status: E5.0 COMPLETE — 5 of 5 predictions confirmed, across two real sessions on 31 August and 1 September. The second closed heart rate by direct measurement and wrist-down behaviour while moving.

E5.1 — Which numbers are recorded, and which are worked out?

Setup: list every number the phone could show after a workout, and classify each one as **recorded**, **derived**, or **not supported**. Then build a review screen that makes the difference visible without explanation.

My prediction

1. **Recorded:** start time, end time, workout duration, active energy, heart-rate samples, and each segment's start offset. These come from HealthKit or from a timestamp written at the moment it happened.
2. **Derived:** every station duration, every roxzone duration, total running time, total station time, and any pace figure. All of these are subtractions or sums that I perform.
3. **Not supported at all:** why a station was slow, whether I am getting fitter, whether pacing was correct, and anything about injury or health. The data cannot reach these and the screen must not imply otherwise.
4. **The line will be blurrier than it looks.** A station duration feels like a recorded fact, but it is the difference between two timestamps I chose to write when I tapped a button. It inherits every mis-tap. I expect the honest category for most of the interesting numbers to be *derived*, not *recorded*.
5. A screen that shows derived numbers in the same style as recorded ones is making a claim it cannot support. **This is the same failure as L4's INTACT verdict**, moved from a test into a user interface.

Actual result — PARTIAL. The screen works; the derived path is not yet demonstrated.

2026-09-01. Classification written to `design/L5-data-honesty.md` (96 lines). Review screen built as `HyroxCoach/WorkoutReview.swift`, the first product surface in the project.

The classification uses four categories, not two. The obvious split is recorded against derived. E5.0 showed that is insufficient:

CATEGORY	MEANING
MEASURED	The device recorded it with no human action — workout start, end, duration, active energy, heart-rate samples
MARKED	A human pressed a button and a timestamp was written — every segment offset. Accurate only if the press was timely
DERIVED	Calculated from the above — every segment duration, every total. Inherits the weakness of its inputs
UNSUPPORTED	The data cannot answer it, and it never appears on screen

A segment offset **is** recorded, but what it records is when a button was pressed, not when the event happened. The 2.03-second roxzone from E5.0 is the proof. So a station duration is DERIVED from the weakest category, twice over.

Observed on device, using a workout from Apple's own app — 31 Aug, 1 h 39 m 53 s, 743 kcal, no HYROX metadata:

- The review screen renders, with **MEASURED** markers on date, total duration and active energy.
- **The markers are orange** — the accent token changed in E5.2. The product screen is visibly consuming the shared token system, which is what E5.2's prediction 2 was missing.
- A workout with no HYROX data **degrades correctly**: "This workout has no segment marks. Only the measured workout summary is available." It is not treated as an error.
- The **"What this cannot tell you"** section is present and permanent, listing all four unsupported claims.

The heart-rate instrument works. The same workout reported **1,169 samples, minimum 77 bpm, maximum 169 bpm, average 118.8 bpm**. The query returns real data at real resolution.

The gap

#	PREDICTION	OUTCOME
1	Recorded: start, end, duration, energy, HR samples, segment offsets	**Confirmed**, but split — the first five are MEASURED, segment offsets are MARKED. The prediction treated them as one category and they are not.
2	Derived: every station and roxzone duration, all totals, any pace	**Confirmed** in the classification. **Not yet seen on screen** — the workout tested had no segments.
3	Not supported: why a station was slow, fitness trend, injury or health	**Confirmed.** All four are named on screen, permanently, and never rendered as data.
4	The line is blurrier than it looks; most interesting numbers are derived, not recorded	**Confirmed, and sharper than predicted.** The blur is not between recorded and derived. It is inside "recorded": MEASURED and MARKED look identical in the data and differ entirely in how far they can be trusted.
5	A screen showing derived values in the same style as recorded ones makes a claim it cannot support	**Held as the design rule.** Enforced by category markers and by rendering segment durations to whole seconds, since their inputs are button presses.

Bonus finding: Apple's own metadata contains an impossible value

The same workout carried `HKWeatherHumidity: 4900 %`. Humidity cannot exceed 100 %. The value is almost certainly 49.00 % stored with a scaling factor the instrument renders raw.

This is a **MEASURED** value — recorded by the device with no human involvement — and it is still wrong on its face. It arrived from Apple, not from this app.

That extends the cycle's argument further than intended. MEASURED was defined as the most trustworthy category. It is the most trustworthy, and it is still not automatically correct: a unit or scaling error produces a number that is complete, well-formed, plausible to a checker, and false. The same shape as the 2.03-second roxzone, arriving from the opposite end of the pipeline.

Verified on a workout with segments, 2026-09-01

A fresh tap-through session with metadata attached was opened in Review. Reported by the learner:

CHECK	RESULT
Segment rows render with durations	**Confirmed**
FROM MARKED TIMES marker appears on values derived from button presses	**Confirmed**
MEASURED distinguishable from DERIVED **at a glance**, without reading markers	**Not reported**

The first two close the mechanical questions: the DERIVED path renders, and the provenance marker reaches the screen rather than only existing in the code.

The third is the actual prediction and remains open. Prediction 5 held that a screen showing derived values in the same style as recorded ones makes a claim it cannot support. Whether this screen avoids that is a question about perception, not about code, and it can only be answered by someone looking at it without being told what to look for. It is recorded as unanswered rather than assumed from the markers being present — the markers existing proves the information is *available*, not that it is *communicated*.

Note on the workout used. This was a tap-through, not real exercise. That is appropriate here: E5.1 asks whether the screen distinguishes categories, which does not depend on the effort behind the numbers. E5.0 already supplied the real session, and its 2.03-second roxzone remains the worked example in `design/L5-data-honesty.md`.

Unplanned finding, 2026-09-01 — RETRACTED. The workouts were not deleted.

While attempting the outstanding E5.1 check, the workout list jumped from **31 Aug 12:57** straight to **28 Aug**. Every workout written by HyroxCoach was absent — 11:17, 10:50, 09:29 on 31 August, and everything from 26 and 27 August. Workouts written by other sources (Apple's Workout app, Liftoff, the Watch's own Running app) were untouched.

This is not a query limit. A row cap truncates the oldest entries; it cannot remove entries from the middle of a descending sort.

Apple's Health app was checked at the time and also appeared not to show them.

THIS CONCLUSION WAS WRONG. At 11:21 on the same day, workout `C1F8E587-6013-4AA0-92DF-785AC67A53BD` — the real session from 31 August, 11:17:14 — was open on screen with **identical data**: offsets 5.262 → 172.688, 134 characters, seven segments, 33.43 kcal. Every figure matches what was logged for it. The workout had not been deleted. It was temporarily not returned.

Cause of the temporary absence: unknown. Possibly a HealthKit sync state, an authorisation hiccup, or a transient query failure. It has not been established and should not be guessed at a second time.

How the error was made. A query returned 20 workouts, none of them from this app, and the list jumped from 31 August 12:57 straight to 28 August. From that absence, deletion was concluded — and written into the workbook and into an architecture decision record as a serious risk.

This is the project's own recurring failure, committed by me, for the second time. The first was a malformed grep that returned zero matches and was read as proof a rationale was missing. This is the same shape: **a null result treated as evidence of absence.** A query that returns nothing tells you the query returned nothing. It does not tell you the thing is not there.

The rule that was already written down and not applied: *a tool's output is evidence about the tool as much as about the subject.*

What the instrument said while this was true: *"Query succeeded: 20 workout(s) found."* A green result sitting on top of missing data. This is the tenth instance of the pattern in this project, and it occurred inside the tool built to report honestly about exactly that. A successful query says the question was answered, not that the answer is complete.

Why nothing already logged is invalidated. Every result in this workbook was recorded with its actual numbers at the time — segment offsets, character counts, durations, verdicts — rather than as a reference to data held elsewhere. The measurements survive because they were written down. Had the workbook recorded "see the workout from 11:17", four cycles of evidence would have been lost with it.

What survives the retraction

The durability question is still worth asking, but it is now a question rather than a finding. Whether an app's HealthKit samples are removed when the app is deleted is genuinely unknown and matters to the L4 decision, which assumed the record persists. It has simply not been demonstrated, and this episode did not demonstrate it.

> **ANSWERED 2026-09-04. The workouts survive.** The app was uninstalled and reinstalled into a fresh container; the 31 August and 1 September sessions were still in Apple's Health app. Health was checked before the reinstalled app was opened, because after a fresh install HealthKit authorisation is reset and this app cannot tell "deleted" from "not permitted to read" — both return empty. See the L4 decision record, where the fourth risk is now closed.

> **Second half of the same test, observed the same day.** The data survives the deletion. **The permission does not.** The watch app is embedded in the iOS app, so uninstalling the phone app removed both; on reinstall the watch app could not start a workout session until HealthKit authorisation was granted again.
>
> **Why this is a finding and not an inconvenience.** After a reinstall the workouts still exist, and the app cannot see them until the user re-grants access. If the user declines, HealthKit returns an empty list with no error — indistinguishable from having no workouts at all. This is the E1.0 blindness appearing in an ordinary user situation rather than in a test harness. Any athlete who reinstalls the app meets it.
>
> **Design consequence, carried to the lo-fi.** The review screen needs a distinct state for *permission not granted* that does not look like *you have no workouts*. The app cannot detect the difference by querying, so the state has to be reached by checking authorisation status directly rather than by inferring it from an empty result.

The risk entry in `design/L4-transport-decision.md` has been corrected to say so: an open question to be tested deliberately, not an observed failure.

Also unchanged and worth keeping: the instrument reported *"Query succeeded: 20 workout(s) found"* while returning none of this app's workouts. That remains true and remains the tenth instance of the pattern. A successful query says the question was answered, not that the answer is complete. It is the reason the wrong conclusion was available to reach.

Second real session, 2026-09-01 — the Review screen rendered in full

Workout `FB4A260E-3749-4661-926D-F970A20C24FB`, 10:51:17 → 10:56:29, **5 m 11 s**, 44.97 kcal – **8.7 kcal/min**, matching the 8.8 of the first real session. Heart rate **51 samples, 72–140 bpm, average 111.9**.

SEGMENT	DURATION
Preparing	35.1 s
Run 1	47.3 s
Roxzone 1	34.3 s
Station 1	35.9 s
Run 2	89.8 s
Roxzone 2	25.3 s
Station 2	36.6 s

The screen distinguishes three categories visually, not only by label:

- **MEASURED** – solid accent-coloured pill, on date, total duration and active energy
- **MARKED** – muted grey pill, on every segment's start offset
- **DERIVED** – amber pill with the line *"FROM TWO MARKED TIMES"* beneath, on every duration and total

One case the implementation got right without being told to. Station 2's duration reads *"FROM MARKED + MEASURED"* rather than *"FROM TWO MARKED TIMES"*. Its start is a button press; its end is the end of the workout, which HealthKit measured. The screen therefore reports that this one duration is **less wrong** than the others – it inherits one human timestamp instead of two. That distinction was not specified. It follows from taking the categories seriously.

Finding not predicted by anyone — the station names are neither MEASURED nor MARKED

The learner substituted **burpees** for the SkiErg and **sandbag lunges** for the sled push. The screen displays **"Station 1 – SkiErg"** and **"Station 2 – Sled Push"**.

Those names were never recorded or marked. They come from a **hard-coded protocol constant** – the app assumes station one is a SkiErg because HYROX says so. The name is displayed with exactly the same authority as a measured duration, and it is **wrong**.

This is a fifth category the classification missed: ASSUMED. A value the app supplies from configuration rather than from the world. It fails differently from the other four — it is not imprecise, not stale, not derived from weak inputs. It is simply **stated**, and nothing in the data can contradict it.

It is also the most confidently wrong thing on the screen. A duration derived from two button presses at least carries a marker admitting its provenance. `SkiErg` carries no marker at all, because the app never doubted it.

Consequence: either the interface must let the athlete record what they actually did, or every station name needs an ASSUMED marker saying the app is naming this from the protocol rather than from observation. Recorded as a design requirement, not fixed — the Act phase is over.

Status: E5.1 substantially complete. The classification is written, the screen is built and consumes the shared tokens, the DERIVED path renders and the provenance marker shows. One sub-question is open: whether the distinction reads at a glance.

E5.2 — One visual system across two screens

The Success Criteria card set four testable conditions for this. None has been attempted.

Setup: define colour, type scale, spacing and iconography once in a shared Swift package used by both targets, then verify by changing a single value.

My prediction

1. **Changing one token will visibly change both apps** with no other edit. If it does not, there are two systems that currently happen to match.
2. **No hard-coded colour or font literal will remain** in either target's view code once the package exists.
3. **The same role will need different values on each device.** A type scale that works on a phone will be too small on a 41mm watch face read mid-burpee, so the package must carry per-platform values rather than one set.
4. **The watch is the harder constraint and should be designed first.** Anything legible on the watch will work on the phone; the reverse is not true.
5. **Least confident:** whether a single shared package can express the platform differences cleanly, or whether it will need enough conditional code that it stops being one system in any meaningful sense.

Actual result — one token changed, both apps changed

2026-08-31. Tokens defined in `Shared/DesignTokens.swift`, compiled into both targets.

The test. The accent colour was changed from blue to the signal orange of the app icon:

```
light: (0.00, 0.42, 0.64)  ->  (0.83, 0.33, 0.12)
dark:  (0.18, 0.78, 0.96)  ->  (0.96, 0.50, 0.31)
```

One file touched. 5 insertions, 2 deletions — and 3 of the insertions were the comment recording the change. Rebuilt, installed, and the accent turned orange on **both** the watch and the phone. No other edit anywhere.

The gap

#	PREDICTION	OUTCOME
1	Changing one token will visibly change both apps with no other edit	**Confirmed.** One value, one file, both apps.
2	No hard-coded colour or font literal will remain in either target's view code	**Partially.** `StylePreview` contains zero literals. The experiment screens contain none either, but only because they use plain system defaults — they do not consume the tokens at all. **The claim is not yet fully tested, because no product screen exists to test it on.**
3	The same role will need different values on each device	**Confirmed.** Watch `body` 18pt against phone 17pt, `caption` 15pt against 13pt. The watch carries proportionally larger text because it is read at arm's length while moving.
4	The watch is the harder constraint and should be designed first	**Upheld**, and evidenced by prediction 3: the watch drove the sizes and the phone accepted them.
5	Least confident: whether one package can express platform differences cleanly, or need so much conditional code it stops being one system	**Wrong, in the useful direction.** It took a **single** `#if os(watchOS)` block inside the token definitions. Callers never branch. It is still recognisably one system.

What is proven: the tokens are genuinely shared. A single edit reaches both platforms, and the per-platform sizing is resolved inside the definitions rather than at the call site.

What is not: that the *product* uses them. The proof surface is `StylePreview`, which exists to demonstrate the tokens. The diagnostic screens were deliberately left alone — they are the harness E5.0 still needs, and restyling them to make a preview look better would have risked the last real experiment in the project.

So the honest status is: the mechanism works, the application of it has not started. The review screen in E5.1 is the first product surface that should consume these tokens, and it is waiting on real data from E5.0.

Order of work: E5.0 first. It produces the only real data in the project, and both E5.1 and E5.2 are about presenting real data honestly. Building a review screen from seeded numbers would repeat this project's most persistent mistake — testing the instrument instead of the thing.

CYCLE L5 · RESULTS SUMMARY

Status: COMPLETE. Run 2026-08-31 and 2026-09-01, ahead of its Sep 3–4 slot. The final cycle of the Act phase.

Conditions — Apple Watch Ultra 3 (watchOS 26.6), iPhone 16 Pro Max (iOS 26.6.1). One session was real physical exercise; the rest were tap-throughs, and each is labelled as such.

The question, and the answer

> What can this data honestly say, and what can it not?

Less than it looks, and the reason is not where I expected. The obvious split is between numbers that were recorded and numbers that were calculated. That split is wrong. The real division sits *inside* "recorded": a workout's duration and a station's start time are both timestamps, but one came from a sensor and the other from a person pressing a button while out of breath.

Results by experiment

EXPERIMENT	QUESTION	RESULT
E5.0	A real training session	**3 m 47 s at 8.8 kcal/min.** First INTACT on genuine exercise. A double-tap produced a 2.03 s roxzone that passed every check.
E5.1	Which numbers are recorded, which are worked out?	**Four categories, not two.** Classification written; review screen built as the first product surface. One sub-question open.
E5.2	One visual system across two screens	**One token edit changed both apps.** Per-platform sizing took a single conditional block.

The four categories

CATEGORY	MEANING	HOW IT FAILS
MEASURED	The device recorded it unaided — start, end, duration, energy, heart rate	Sensor error, or a unit mistake. Apple's own metadata reported humidity of **4900 %**
MARKED	A human pressed a button and a timestamp was written — every segment offset	A late, early or accidental press. Nothing in the data reveals it
DERIVED	Calculated from the above — every segment duration, every total	Inherits every weakness of its inputs, twice over when both are MARKED
UNSUPPORTED	The data cannot answer it	Never appears on screen. Naming it is how it is kept out

---

The two numbers that carry the cycle

2.03 seconds. The second roxzone of the real session, caused by tapping Advance twice by accident. Physically impossible — the first roxzone in the same session took 24.76 s. The integrity check passed it, because 2.03 is non-negative, in order, and inside the workout's duration. **Plausible by every rule and false in fact.**

4900 %. The humidity Apple recorded on a different workout. A MEASURED value — the most trustworthy category — produced with no human involvement, and impossible on its face.

Together they close the argument from both ends. A number can be complete, well-formed, correctly transported, verified by a checker, and wrong. The failure arrives from human timing at one end and from unit handling at the other, and no amount of interface honesty repairs either.

Prediction tally

EXPERIMENT	CONFIRMED	BY PROXY	OPEN / UNTESTED
E5.0 (5)	3	1	1 – wrist-down while moving
E5.1 (5)	4	0	1 – whether the distinction reads at a glance
E5.2 (5)	3	0	1 partial, **1 wrong favourably**

E5.2's wrong prediction was pessimism again: I expected platform differences to need so much conditional code that the token system would stop being one system. It took a single `#if os(watchOS)` block, and callers never branch.

What L5 built

- `design/L5-data-honesty.md` – the classification, with the 2.03-second roxzone as the worked example
- `Shared/DesignTokens.swift` – colour, type, spacing and icons defined once, per-platform values resolved inside the definitions
- `HyroxCoach/WorkoutReview.swift` – the first product screen in the project. **71 token references, zero literal colours, sizes or padding values.** Segment durations render to whole seconds because their inputs are button presses. A permanent, non-dismissible section states what the app cannot tell you.

The unplanned finding, and it is the most serious in the project

On 2026-09-01, **every workout written by this app was found to be deleted from HealthKit**, confirmed absent in Apple's own Health app. Workouts from other sources on the same device were untouched. The cause was not determined and is recorded as unexplained; the likely explanation is that iOS removes an app's HealthKit samples when the app is deleted, and this project reinstalled repeatedly.

Why it matters beyond the lost test data. L4 decided against building a transport because HealthKit already carries the record. That decision is sound on its evidence – but it assumed the record *persists*. If an app's samples die with the app, an athlete's entire history depends on never deleting it. That is a durability problem, not a transport problem, and it would need its own design. Added to the L4 decision record as a fourth risk and a reversal condition.

Why nothing already logged was lost. Every result in this workbook carries its actual numbers rather than a reference to data held elsewhere. Four cycles of evidence survived an event that deleted the data they were measured from.

The pattern, complete

L1 recorded six instances of a success signal concealing a failure. L2 added four, inside the instruments built to catch them. L4 added one more, in the integrity check itself. L5 adds the tenth: **"Query succeeded: 20 workout(s) found"**, displayed over a list with every one of this app's workouts missing.

A successful query says the question was answered. It does not say the answer is complete.

CYCLE L6 — the number, and what happened to it

The original L6 was a presentation wrapper. It was dropped in the 2026-08-24 re-baseline, because a cycle producing no learning did not deserve one of the ten Act days.

The number was reused on 2026-09-03 for the platform-idiom work, which did produce learning: how Apple records structure, a migration to `HKWorkoutEvent`, two defects that migration exposed, and whether a workout can carry the name HYROX.

That cycle is recorded below, after the L5 summary and before the failure log.

L1 VERIFICATION PASS — 2026-08-21

An independent consistency check of every recorded L1 number, run against the source code rather than against the photographs. This does **not** re-run the experiments; it checks whether the recorded data is internally coherent and consistent with what the code can produce.

Code model confirmed. Both probes use `Timer.scheduledTimer(withTimeInterval: 1, repeats: true)`, and `elapsedByTicks` accumulates per tick while `elapsedByDate` is computed from a stored `startDate`. So one tick must equal exactly 1.0 s of counted time, and the two figures are genuinely independent measures rather than two views of the same counter.

CHECK	RESULT
Ticks equal counted seconds — E1.1, E1.2 A, E1.2 B	PASS (20/20.0, 1134/1134.0, 1610/1610.0)
Divergence arithmetic — all three readings	PASS (77.0, 15.0, 20.0 s)
Proportion lost — all three readings	PASS (79.38 %, 1.305 %, 1.227 %)
Inter-reading window A→B: 481 s elapsed, 476 ticks	PASS — 5.0 s lost, 1.04 %
Extrapolation to a 90-minute race at reading B's rate	PASS — 66.3 s
E1.3: start 10:19:35 against recovery at ~10:21	PASS — ~85 s, consistent with a force-quit and relaunch

Verdict: all consistent. No arithmetic error, and no figure that the code could not have produced.

What this verification does NOT establish. It confirms the recorded numbers are coherent. It cannot confirm they were observed under the stated conditions — that Always-On was off, that no debugger was attached, that the wrist was actually down. Those are physical facts about the room, evidenced only by the photographs and by Stanley's account of them. A consistency check cannot substitute for a witness, and claiming otherwise would repeat this cycle's central error at the level of the record itself.

CYCLE L6 · The platform's own idiom — 2026-09-03

Learning question: *Is the way I record a workout's structure the way the platform records it — and if not, should I change?*

How this cycle differs from the other five, and it matters. L1–L5 were planned, with predictions written before every experiment. **L6 was not.** It began as a follow-up to a conversation with a technical mentor, who asked why `HKWorkoutEvent` was not being used. The work was investigative rather than predictive: questions were asked of the platform and answered from its data, but no predictions were recorded in advance.

That is a real departure from the method the rest of this workbook follows, and it is recorded rather than disguised. The findings below are evidence; they are not scored against anything, because there was nothing to score them against.

E6.0 — Does Apple's own Workout app record `HKWorkoutEvent` structure?

Why this is being asked. This project records station boundaries as a custom metadata string. HealthKit has a first-class concept for intervals inside a workout — `HKWorkoutEvent`, with `.segment`, `.marker` and `.lap` types — which was never used. A technical mentor raised it, and the fastest way to judge it is to look at what Apple itself writes rather than argue about it.

The instrument. The iOS detail view now lists every event on a workout: readable type name, start offset **relative to the workout's start** so it is directly comparable with this project's own segment offsets, duration, and each event's own metadata. An empty list reports `NO EVENTS` and is presented as normal, because most apps record no structure.

Result — inconclusive, and deliberately recorded as such.

Apple's own Workout app, Running, 3 Sep 10:06:57, 3 m 5 s:

```
Workout events: NO EVENTS
This workout carries no events. This is normal for apps that do not
record structure.
```

What this establishes: the instrument works, and a plain Apple run with no laps pressed carries no events.

What it does not establish: whether Apple writes events *when there is structure to write*. This workout had none. `HKAverageMETs 1.6` and a 71–88 bpm range show it was started and stopped at a desk, so no lap button was pressed either.

Concluding from this that Apple never records structure would be the same error made twice already in this project — reading an absence as proof. The test is not finished. It needs either the Lap button pressed several times, or a custom interval workout built in Apple's Workout app, and then the events checked again.

RESOLVED 2026-09-03, 10:26. A second run in Apple's Workout app with the Lap button pressed produced:

```
Workout events: 5 EVENTS · 5 marker

marker  start offset  24.931 s   duration 0.000 s
marker  start offset  51.251 s   duration 0.000 s
marker  start offset  73.555 s   duration 0.000 s
marker  start offset  73.888 s   duration 0.000 s
marker  start offset 102.251 s   duration 0.000 s
```

Apple does record workout events. It uses `.marker`, not `.segment`.

Every duration is 0.000 seconds. Markers are *points in time*, not intervals. Apple does not record "this lap lasted 26 seconds" — it records "something happened at 51.251 s" and leaves the arithmetic to whoever reads it.

That is structurally identical to what this project already does. `HYROXSegments` stores a start offset per segment and derives every duration by subtraction. Apple's own app takes the same approach through a first-class API instead of a parsed string.

The consequence for the architecture. The choice is not between "my way" and "Apple's way" — they are the same model. It is between two places to put it:

	`METADATA STRING (CURRENT)`	`HKWORKOUTEVENT`
Readable by other apps	No	Yes
Parsing required	Yes, split on `;` and `@`	None
Size ceiling	Unknown, untested	Not a concern
Silent truncation risk	Real	None
Carries a name per boundary	Yes, in the string	Needs `.segment` with metadata, or a parallel scheme

Apple choosing `.marker` does not prevent this project writing `.segment` events with a real `dateInterval` and the station name in each event's own metadata — which would be strictly better than both, and is available through `HKWorkoutBuilder.addWorkoutEvents`.

The detail worth more than the answer

Markers 3 and 4 are **0.333 seconds apart**: 73.555 and 73.888.

That is a double press. The Lap button was hit twice in a third of a second, and **Apple's own Workout app recorded both**, with no debounce, no merge, and nothing marking the second as suspect.

This is exactly the failure that produced this project's 2.03-second roxzone in E5.0 — a mis-tap while moving, recorded as fact. It appears here in Apple's own first-party app, written by the team that owns the API.

So the correction problem is not a beginner's mistake in this project. It is unsolved in the platform's reference implementation. Which raises the design bar rather than lowering it: if Apple has not solved marking boundaries accurately mid-effort, doing it well is a genuine contribution rather than catching up.

Status: RESOLVED. Apple records structure as markers. `HKWorkoutEvent` is a legitimate and better home for segment boundaries than a metadata string. Migration is a change to *where* structure is written, not *how* it is modelled.

E6.1 — Apple's own MEASURED values carry a systematic unit error

E5.1 recorded one workout reporting `HKWeatherHumidity: 4900 %`, and noted that humidity cannot exceed 100 %.

A second workout reported `HKWeatherHumidity: 6000 %`, and a third `5900 %`. Different sessions, same impossible shape. Almost certainly 49.00 %, 60.00 % and 59.00 % stored with a scaling factor the instrument renders raw.

Three data points make it systematic rather than an anomaly. This matters to the classification in `design/L5-data-honesty.md`:

MEASURED was defined as the most trustworthy category — recorded by the device with no human involvement. It is still the most trustworthy. It is also **reliably wrong in this field**, and nothing in the value signals that. A unit or scaling error produces a number that is complete, well-formed, correctly transported, and false — arriving from Apple rather than from anything this project wrote.

Consequence: MEASURED means *"no human timed this"*, not *"this is correct"*. Any value from an outside source needs a plausibility bound the same way a MARKED value does — humidity cannot exceed 100 %, a heart rate cannot be 400, a segment cannot start before its workout. The plausibility check built in L4 for this project's own data applies just as much to data it merely receives.

E6.2 — Migration to `HKWorkoutEvent`, verified — and the bug it exposed

2026-09-03, 10:52. Segments now written as `HKWorkoutEvent` of type `.segment` alongside the existing metadata string, so the two can be compared before the string is retired.

Result: `AGREE — 7 segment(s)`. Seven `.segment` events, each with a real `dateInterval` and metadata carrying `HYROXSegmentName` and `HYROXSegmentIndex`. `HKWorkoutBrandName: HYROX` also present.

The events are internally consistent. Each duration chains exactly into the next segment's start:

```
Preparing           4.843 + 0.933 =  5.776  = next start
Run 1               5.776 + 0.949 =  6.725  = next start
Roxzone 1           6.725 + 0.667 =  7.392  = next start
Station 1 — SkiErg  7.392 + 1.033 =  8.425  = next start
Run 2               8.425 + 0.750 =  9.175  ≈ next start 9.174
Roxzone 2           9.174 + 1.599 = 10.773  = next start
Station 2           10.773 + 0.600 = 11.373  ← protocol ends here
```

Unlike Apple's markers, which report `0.000`, these carry genuine durations.

The bug: the two representations disagree about the final segment

	STATION 2 — SLED PUSH
`.segment` event duration	**0.600 s**
Review screen duration	**9 s**

The workout ran to **19 s**, but the protocol's last segment ended at **11.373 s** — 7.6 seconds before the workout was stopped.

`WorkoutReview` computes the final segment's duration as *workout end minus segment start*, and labels it **"FROM MARKED + MEASURED"** — treating the workout's end as the segment's end. That is only correct if the workout is stopped at the instant the last station finishes. Here it was not, and the screen reported a 9-second station that actually lasted 0.6 seconds.

The event is right and the screen is wrong. The segment genuinely ended when Advance was tapped into `completed`; the extra 7.6 seconds is time after the protocol, not part of the station.

Why the check said AGREE anyway

The comparison tests **count, names and start offsets**. It does not compare durations — because until this change only one representation had them.

So `AGREE` is **true for what it compared**, and it does not mean the two representations agree about everything. That is the project's recurring pattern once more, and this time it appeared in the verification built for the migration itself, on the first run.

It is also the argument for having migrated. The string carried only start offsets, so this disagreement could not exist in it — and therefore could not be *detected* in it. Adding a second representation with real end times made a wrong number on the review screen visible. The bug was always there; nothing could see it before.

Two actions follow:

1. Extend the comparison to include durations, so `AGREE` means what a reader assumes it means.
2. Fix `WorkoutReview` to use the segment's own end rather than the workout's, and reserve "FROM MARKED + MEASURED" for the case where those genuinely coincide.

Both defects fixed, 2026-09-03.

- `WorkoutReview` now takes each segment's duration from its event's own `dateInterval`. A segment with no recorded end displays as **unknown** rather than absorbing whatever time remained after the protocol finished. **"FROM MARKED + MEASURED"** is now used only when the segment's end genuinely coincides with the workout's, within one second.
- The agreement check now compares **durations** as well as count, names and offsets, at the same 0.05 s tolerance. The final segment's duration cannot be derived from the string, so that is **stated in the verdict** rather than skipped silently: `AGREE — 7 segment(s), 6 durations compared; final segment duration skipped because the string has no following entry.`

Confirmed on device 2026-09-04. The fixed iOS build was installed on the iPhone and an existing 3 Sep workout was opened. The agreement line now reads:

> `AGREE — 6 durations compared`

That is the fix working. The old line said `AGREE` and meant only that the segment *names* matched; it compared no durations at all and said nothing about it. A reader had no way to tell the difference between "the durations agree" and "the durations were never checked". Both printed the same word.

What this confirms: durations are read from each event's own `dateInterval`, compared at 0.05s tolerance, and the count of comparisons is stated. The check now reports its own coverage.

What is still unconfirmed: the two remaining changes to the review screen — that per-segment durations render from `event.duration` rather than from the leftover remainder, and that `FROM MARKED + MEASURED` is withheld when the final segment does not end with the workout. Neither has been read off a screen yet.

Status: E6.2 complete. Migration verified, both defects fixed, the agreement fix confirmed on device.

E6.3 — Can a workout carry the name HYROX?

Why. `activityType` is a fixed Apple enum with no HYROX case, so every session is permanently recorded as Cross Training. `HKMetadataKeyWorkoutBrandName` is the only place a workout can carry a name that enum does not cover.

Result so far. The key is written and arrives: the 10:52 session shows `HKWorkoutBrandName: HYROX` in its metadata on the phone.

RESOLVED 2026-09-03. Apple's Health app displays the workout as HYROX.

The brand name is not a private label. It reaches Apple's own interface, on a workout recorded entirely by this app, with this app's own live screens.

What this does and does not change — the distinction matters.

Presentation	The workout is displayed as **HYROX** in Apple's Health app
Classification	It is still `.crossTraining` internally

`activityType` remains the fixed enum it always was. Rings, activity categorisation and energy estimation still treat this as cross training, because that is what the type says. The brand name changes what a person reads; it does not change what the system counts it as.

Why this is the answer to the problem as posed. The question was whether a workout can be called HYROX when HealthKit has no HYROX type. It can — visibly, in Apple's own app, at the cost of one metadata key. The alternative route, WorkoutKit's `CustomWorkout` with a `displayName`, would have achieved the same naming while surrendering the live watch experience entirely. One line of metadata bought what an architectural rewrite was being considered for.

Status: E6.3 complete.

RUNNING FAILURE LOG

Every bug, with the symptom, the layer it *appeared* to be in, and the layer the cause was *actually* in. The mismatches are the most valuable rows.

DATE	SYMPTOM	LOOKED LIKE	ACTUALLY WAS	FIX	
Aug 20	`xcodebuild` refuses to run; no iOS/watchOS SDKs listed	Xcode not installed	Xcode IS installed at /Applications/Xcode.app, but `xcode-select` pointed at the Command Line Tools instance instead	`sudo xcode-select -s /Applications/Xcode.app/Contents/Developer`	
Aug 20	"No profiles for 'com.stanleyyoung.HyroxCoach.watchkitapp' were found"	Bundle ID or signing config wrong	Program License Agreement was unaccepted on the developer account, so the team could not issue any profile at all	Accept the updated PLA at developer.apple.com/account	
Aug 20	Same "no profiles" error, after the PLA was accepted	Still a signing config problem	Team had **no registered devices**; a development profile must name specific devices	Connect and trust iPhone, register both devices with the team	
Aug 20	"Your team has no devices" even with both devices connected	Devices not detected by the Mac	They *were* detected — but `generic/platform=watchOS` names no specific device, so there was nothing concrete to register	Build against `platform=watchOS,id=<device-id>` instead of the generic destination	
Aug 20	`devicectl` install reported shell exit code 0	Install succeeded	Install had **failed** (`IXRemoteErrorDomain error 6`); the exit code came from the shell, not the install. The old build was then launched instead	Read the actual output, not the exit status. Reinstalled from Xcode	
Aug 21	E1.1 ticks never stopped with the wrist down	watchOS is more permissive than predicted	The **debugger was attached** after installing from Xcode. A debug session prevents suspension, so the experiment measured the debugger, not the OS	Stop the Xcode session; launch from the watch itself	
Aug 21	`recoverActiveWorkoutSession` threw "endpoint already exists"	Recovery is broken on this watchOS version	Initially read as a malformed test. On repetition it proved **reproducible**: recovery reliably fails after a force-quit while a workout is active. The one-off explanation was wrong	Open question for L2 — see the E1.3 amendment	
Aug 24	Audit grep reported a missing rationale in the design	The design had a documentation gap	The **grep pattern was malformed** — `\` `alternation used with` -E`. It matched literally and returned 0. The rationale was present all along	Verify patter match somet known preser before trustir zero re	
Sep 4	The iPhone's "Design system" heading looked slightly wrong and the preview was always on screen	A styling problem, or a deliberate choice made earlier	A `Section` was nested **inside a `VStack`, outside any `List`**, with `StylePreview` embedded inline instead of linked. `Section` outside a container is not an error — SwiftUI renders it as a plain stack, so it looked almost right	Move it into the `List` as a `NavigationLink`	
Sep 4	Tapping "Style preview" on the watch did nothing	The preview screen was broken, or the tap was not registering	The **link was never connected to anything.** There was no `NavigationStack` anywhere in the watch app, so every `NavigationLink` rendered as a normal tappable row and navigated nowhere. The destination view was fine and had simply never been reachable	Wrap the root list in a `NavigationStack`. Confirmed working on device	

Note on the two Sep 4 rows. These are the **twelfth and thirteenth** instances of the pattern. Both are layout mistakes that produced *something* on screen. Neither raised a warning, and neither looked broken enough to investigate. A `Section` outside a `List` renders as a plain stack; a `NavigationLink` without a `NavigationStack` renders as a tappable row. **SwiftUI has no failure state for either — it degrades quietly into something plausible.** Both were found while restructuring the apps for readability, not by anything reporting them, and both had been in place for the entire Act phase.

Note on the dead-link row. This is the twelfth instance of the pattern this project keeps finding, and the purest one. A `NavigationLink` with no navigation container does not warn, does not log, and does not look disabled. It draws the chevron, highlights on touch, and goes nowhere. Nothing distinguishes it from a working link except tapping it and noticing that the screen did not change — which requires already suspecting it. It was found only because the app was being restructured for readability, not because anything reported it.

Note on the first row: this is exactly the class of failure L1 is about — the symptom pointed at one layer (no Xcode) and the cause was in another (toolchain selection). Worth keeping as the template for how rows in this table should read.

PREDICTION SCORECARD

Every experiment in the Act phase, in order — 19 across five cycles, plus two predictions tested separately. The four L6 rows are post-Act and carry no predictions, which is recorded rather than hidden.

EXPERIMENT	PREDICTION HELD?	NOTE
E1.0	6 of 6 confirmed	Round trip PASSED, but over-claimed — see E1.0b amendment. Prediction 6 (provisioning is the time sink) was the strongest hit.
E1.0b	CONFIRMED	Added after review found the round trip could not evidence read access. Read access now separately proven.
P2	Confirmed by sabotage test	Builds clean, crashes instantly on authorise. `catch` never runs.
P5	Confirmed by denial test	Denial is invisible: no error, empty result. The side-prediction that E1.0b would flip to INCONCLUSIVE was WRONG — the aggregate verdict masked it. Probe since fixed and re-verified: reports MIXED on the same case.
E1.1	5 of 5 confirmed	20.0 s counted vs 97.0 s real — 77 s lost. First attempt invalid: debugger attached.
E1.2	2 confirmed, 1 refined, 2 untested	GATE PASSED. 1.23% lost vs 79.4% without a session. Two readings show the drift is steady, not a start-up artifact.
E1.3	Recovery possible; launch path solved in L2	Reproducible three-state sequence. Recovery after force-quit reliably FAILS while a workout is active. **Cause identified by E2.0b:** the call is not idempotent and fails whenever the process already holds a session.
E2.0	Prediction 1 FALSIFIED	The system does NOT relaunch a crashed app holding a workout session. Kills the only explanation for E1.3's endpoint conflict — cause now unknown.
E2.0b	3 confirmed, 1 refined, 1 unneeded	NOT IDEMPOTENT. Call 1 succeeds, call 2 conflicts with call 1's own session. Explains every E1.3 observation and settles L2's launch path.
E2.1	3 confirmed, 1 mixed · audited	v1 design: 10 states, 26 transitions. Awkward paths lose on state count (4:6) but win on transitions (20:6). Complexity lives in edges, not states.
E2.2	1 confirmed, 1 wrong, 3 untested	Nothing lost across a force-quit: segments identical, elapsed carried through the dead time. Atomicity and the lossy window remain unproven.
E2.2b	***5 of 5* confirmed**	Atomic at all three crash points. Round 3's complete 3239 B temp correctly ignored. Harness bug found mid-run via an anomalous byte count; two earlier scorings corrected.
E2.3	2 confirmed, 1 wrong on magnitude, 1 design	ROUND-TRIP EXACT — offsets remove reconciliation. Delta was 11.4s of operator delay, not sub-second drift. Found that start() wipes the HK link.
E3.0	4 confirmed, **1 falsified (favourably)**	Crosses in ≤14s with no transport code — even with the phone LOCKED and in another room. The conditional-crossing worry was unfounded.
E3.1	3 confirmed, 1 partial	Envelope arrives intact; zero semantic structure. Only metadata is Apple's HKIndoorWorkout. HR samples unverified.
E3.2	3 confirmed, 1 by construction, **1 wrong (it worked)**	All four HYROX keys crossed intact with no transport code. Size limit untested at full race length. L4 now needs rescoping.
E4.0	3 confirmed, 2 not reached	25 segments crossed complete (601 chars, then 543 after the fix). First run: every offset negative and passed as INTACT — completeness is not correctness. Re-run: both fixes verified, check correctly refused seeded data.
E4.1	Not run	Written to run only if E4.0 failed. It did not fail, so finding the true size limit is optional rather than necessary.
E4.2	5 of 5 confirmed	Nothing left to build. ADR written: do not build the transport. Reversal conditions and three accepted risks recorded.
E5.0	***5 of 5* confirmed**	Two real sessions, 31 Aug and 1 Sep, at 8.8 and 8.7 kcal/min. Heart rate measured directly: 51 samples, 72–140 bpm. Wrist-down held for 89.8 s while running. A double-tap produced a 2.03 s roxzone that passed every check.
E5.1	4 confirmed, 1 open · plus an unpredicted fifth category	Four categories, not two: MEASURED and MARKED look identical in data and differ entirely in trust. Screen works and consumes the tokens. DERIVED path not yet seen on screen. Apple's own metadata carried humidity of 4900%.
E5.2	3 confirmed, 1 partial, **1 wrong (favourably)**	One token edit changed both apps. Per-platform sizing took a single conditional block. Proven on the preview surface only — no product screen consumes the tokens yet.
E6.0	no predictions — investigative	Apple records lap presses as `.marker` with 0.000 duration. Points, not intervals — the same model this project already used, in a first-class API.
E6.1	no predictions — investigative	Apple's own humidity reported 4900 %, 6000 % and 5900 % across three workouts. MEASURED is the most trustworthy category and still not automatically correct.
E6.2	no predictions — investigative	Migration verified AGREE on 7 segments. Immediately exposed a review-screen bug the metadata string could not have shown. Both defects fixed; the agreement fix confirmed on device 2026-09-04 (`AGREE — 6 durations compared`).
E6.3	no predictions — investigative	**Apple's Health app displays the workout as HYROX.** Presentation solved with one metadata key; classification is still `.crossTraining` , so rings and energy still count it as cross training.